

Remote Health Monitoring System for the Elderly

Bachelor Graduation Project

EE3L11

Yasmina Aarkoub
Mohamed Almifalani
Frederik Schiepers

Remote Health Monitoring System for the Elderly

Bachelor Graduation Project

by

Yasmina Aarkoub
Mohamed Almifalani
Frederik Schiepers

Supervisor:	P. French
Supervisor:	K. Rassels
Chairman:	R. Van Swaaij
Committee:	K. Dowling
Project Duration:	April, 2026 - July, 2026
Faculty:	Faculty of Electrical Engineering, Delft
Cover:	Image adapted from [1]

Abstract

Population ageing is increasing the need for home-based care while the available care workforce remains limited, creating a need for unobtrusive systems that support routine observation without adding tasks for elderly residents. This thesis presents the design, implementation, and validation of a concept demonstrator for a remote health monitoring system built around a smart mirror and modular sensor printed circuit board. The system measures presence, heart rate, respiratory rate, environmental conditions, and ambient light using radar and supporting sensors. A resident-facing mirror display presents the most relevant information with large visual elements, limited text, and a traffic-light interpretation scheme, while requiring no active user input. Sensor readings are published over an encrypted, certificate-based local communication link to a broker hosted on the tablet behind the mirror. A gateway subscribes to these readings, buffers them during outages, converts vital-sign data into standard healthcare observation resources, and forwards them to a reference healthcare server. Validation showed that the access-control design confined devices to their own data paths and rejected unauthenticated connections, while the custom circuit board and sensor interfaces were brought up successfully. The vital-sign radar followed expected physiological trends, although heart-rate readings showed a systematic overestimation and breathing-rate readings were closer to the reference. The project demonstrates a secure path from ambient home sensing to a healthcare information interface, but further work is needed on clinical-grade validation, production endpoint authentication, deployment hardening, and resident identification.

Contents

Abstract	i
List of Acronyms	iv
1 Introduction	1
1.1 Use of GenAI	2
2 Background Research and Design Choices	3
2.1 Smart Mirror	3
2.1.1 Interface Design for Elderly Individuals	3
2.1.2 Available Software for a Smart Mirror GUI	3
2.1.3 Smart Mirror Architecture Options	4
2.2 Vital Sign Detection Using Radar Sensors	5
2.2.1 Background and Relevance of mmWave Vital Sign Detection	5
2.2.2 Principles of mmWave Vital Sign Sensing	5
2.2.3 State of the Art	6
2.2.4 Commercial Radar Modules for Vital Sign Detection	6
2.2.5 Comparison of mmWave Radar Sensor Modules	7
2.2.6 Presence Detection Radar Sensor Selection	7
2.3 Communication Protocols and API Integration	8
2.3.1 Protocols	8
2.3.2 Security: Truly Static mTLS	8
2.3.3 Simplicity: Keep the Existing Stack	9
2.3.4 Summary of Changes	10
2.3.5 The Two Communication Legs	10
2.3.6 HL7 FHIR	10
2.3.7 Deployment and Open Questions	11
2.4 System Architecture	11
2.5 Future Work: Face Recognition	12
2.5.1 A Self-Contained Vision Module: HuskyLens	12
2.5.2 Decision Logic and Failure States	13
2.5.3 The Two Approaches Compared	13
2.5.4 Privacy and Hardware	14
3 Program of Requirements	15
3.1 Smart Mirror Requirements	15
3.1.1 Sensor Module	15
3.1.2 User Interface	15
3.1.3 Sensor Module Display	16
3.1.4 Communication Between the Sensor Module and the Broker	16
3.1.5 Integration with Existing Health Care IT Infrastructure	16
3.1.6 Deployment Script	16
4 Implementation	18
4.1 System Overview	18
4.1.1 Mirror-paired Sensor Module	19
4.1.2 Generic Sensor Module	19
4.1.3 Tablet	20
4.1.4 Gateway	20
4.1.5 Data Path	20
4.1.6 Linux Environment Deployment Options	21

4.2	User Interface	22
4.2.1	Graphic Design of the User Interface	22
4.2.2	Programming the User Interface	24
4.2.3	Fullscreen User Interface Configuration	24
4.3	Communication	25
4.3.1	Topic Namespace and Message Schema	25
4.3.2	Transport Security and Access Control	26
4.3.3	Sensor-Node Client and Outage Handling	27
4.3.4	Command and Time Synchronisation	28
4.3.5	Gateway Aggregation and FHIR Translation	28
4.3.6	Verification	29
4.4	Automation and Deployment	31
4.4.1	Tablet Deployment Script	31
4.4.2	Display Activation and Deactivation	31
4.5	Mirror Materials	33
4.6	PCB Design and Component Integration	34
4.6.1	Modular Radar Footprint and Antenna Keep-Out Strategy	34
4.6.2	Power Architecture	35
4.6.3	Buck Converter Component Selection	36
4.6.4	Microcontroller, Sensors, and User Interface	36
4.6.5	PCB Layout Considerations	37
4.6.6	Test Points	37
4.7	GL5516 Calibration Method and Result	37
4.8	HMMF Filtering and Usage	38
4.9	Filtering Method for Improving Reliability of MR60BHA2 Vital-Sign Output	40
5	Results and Discussion	43
5.1	Radar Sensing Through Mirror Materials	43
5.1.1	Openings for the Radar Sensor	43
5.1.2	Test Setup	45
5.1.3	Results and Discussion	46
5.2	PCB Test and Validation	47
5.3	MR60BHA2 Validation	50
6	Conclusion	52
	References	54
A	Visuals related to the Printed Circuit Board (PCB)	57
B	Ethical Analysis	61
B.1	Societal Impacts	61
B.2	Ethical Aspects of Development	61
B.3	Ethical Evaluation	61
B.4	Reflection on Design Choices	61
B.5	Forward-looking Responsibility	62
C	Project Proposal	63
D	Tablet Deployment Instructions	65
D.0.1	A two-stage procedure	65
D.0.2	On-device provisioning (deploy.sh)	65
D.0.3	Self-healing autostart (tablet_boot.sh)	66
D.0.4	Network-independent broker discovery	66
D.0.5	Manual steps (Android restrictions)	66
D.0.6	Summarised procedure	66

List of Acronyms

Acronym	Definition
ADC	Analogue-to-Digital Converter
AES	Advanced Encryption Standard
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
AQI	Air Quality Index
BAP	Bachelor Graduation Project (Bachelor Afstudeerproject)
bpm	Beats per minute
BR	Breathing Rate
BRSKI	Bootstrapping Remote Secure Key Infrastructure
CA	Certificate Authority
CBC	Cipher Block Chaining
CBOR	Concise Binary Object Representation
CN	Common Name
CoAP	Constrained Application Protocol
CPU	Central Processing Unit
CSR	Certificate Signing Request
CW	Continuous Wave
DNS	Domain Name System
ECDHE	Elliptic-Curve Diffie–Hellman Ephemeral
ECDSA	Elliptic Curve Digital Signature Algorithm
eCO₂	Equivalent Carbon Dioxide
EMA	Exponential Moving Average
FFT	Fast Fourier Transform
FHIR	Fast Healthcare Interoperability Resources
FIFO	First In, First Out
FMCW	Frequency-Modulated Continuous Wave
GCM	Galois/Counter Mode
GenAI	Generative Artificial Intelligence
GND	Ground
GPIO	General-Purpose Input/Output
GUI	Graphical User Interface
HDMI	High-Definition Multimedia Interface
HL7	Health Level Seven
HMMD	Human Micro-Motion Detection
HR	Heart Rate
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
I²C	Inter-Integrated Circuit
IAQ	Indoor Air Quality
IDevID	Initial Device Identifier
IETF	Internet Engineering Task Force
IoT	Internet of Things
IP	Internet Protocol
IR-UWB	Impulse-Radio Ultra-Wideband

Acronym	Definition
IT	Information Technology
JSON	JavaScript Object Notation
LCD	Liquid-Crystal Display
LDR	Light-Dependent Resistor
LED	Light-Emitting Diode
LOINC	Logical Observation Identifiers Names and Codes
LVGL	Light and Versatile Graphics Library
MAD	Median Absolute Deviation
MCU	Microcontroller Unit
mDNS	Multicast Domain Name System
mmWave	Millimetre Wave
MQTT	Message Queuing Telemetry Transport
MQTT-SN	Message Queuing Telemetry Transport for Sensor Networks
mTLS	Mutual Transport Layer Security
NTP	Network Time Protocol
OCSP	Online Certificate Status Protocol
OLED	Organic Light-Emitting Diode
PCB	Printed Circuit Board
PCR	Pulsed Coherent Radar
QoS	Quality of Service
REST	Representational State Transfer
RFC	Request for Comments
RISC-V	Reduced Instruction Set Computer, fifth generation
RSA	Rivest–Shamir–Adleman
Rx	Receive
SBC	Single-Board Computer
SCL	Serial Clock Line
SDA	Serial Data Line
SDK	Software Development Kit
SHA	Secure Hash Algorithm
SMART	Substitutable Medical Applications, Reusable Technologies
SNOMED CT	Systematized Nomenclature of Medicine – Clinical Terms
SPI	Serial Peripheral Interface
SSH	Secure Shell
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TVOC	Total Volatile Organic Compounds
Tx	Transmit
UART	Universal Asynchronous Receiver-Transmitter
UCUM	Unified Code for Units of Measure
UDP	User Datagram Protocol
UI	User Interface
URN	Uniform Resource Name
USB	Universal Serial Bus
UUID	Universally Unique Identifier
UWB	Ultra-Wideband
WAN	Wide-Area Network

1

Introduction

The population of the Netherlands, in common with most high-income countries, is ageing: the share of residents above 65 is rising faster than the working-age population that supports them [2]. The government encourages (supported) independent living, keeping older adults in their own homes for longer rather than moving them into residential care [3]. That choice quietly relocates a task that used to happen by default. In a care home, the simple fact of staff moving through the building means that an older adult's presence, activity, and apparent wellbeing are observed continuously and informally. At home, that observation has to be reconstructed deliberately, and it falls on family caregivers and an already strained community-care workforce. The gap this opens is not a shortage of acute treatment but a shortage of routine observation: knowing whether an older adult is present, active, and physiologically stable, without requiring that person to operate, wear, charge, or even think about any device.

Several classes of home-monitoring technology already exist, and each pays a characteristic price for the information it provides. Personal emergency response systems, the wearable alarm button, are purely reactive and depend on the user being conscious, able, and willing to press them. Wrist-worn trackers measure heart rate and activity but must be worn, charged, and operated daily, which is precisely the demand that motor and cognitive decline make difficult to meet, so adherence falls. Camera-based monitoring is information-rich but is the most privacy-invasive option and is poorly tolerated in living spaces, especially the bedrooms and bathrooms where falls are common. Ambient contact sensors such as bed, floor, and door sensors are unobtrusive but infer activity only indirectly. Millimetre-wave radar occupies a useful position against this landscape: it detects human presence and estimates respiration and heart rate from sub-millimetre chest motion, contactlessly, without a worn device and without producing any photographic image of the resident [4]. Useful physiological information at no device burden and with no camera record is the property that makes radar attractive for an elderly population, and it is the sensing principle adopted in this work.

The system developed in this thesis is a remote health-monitoring kit for a single home, organised as three on-premises tiers that deliver to an external health care provider. At the edge, a microcontroller-based sensor module samples a presence and vitals radar together with environmental and light sensors and drives a small on-device status display, acting only as a thin publisher that formats each reading as a compact message and forwards it. A tablet mounted behind a two-way mirror serves two roles at once: it hosts the local message broker through which all sensor traffic passes, and it runs the resident-facing display, so that the same surface is an ordinary mirror when idle and becomes a health display when someone stands in front of it. A gateway aggregates the readings, translates them into the standard clinical exchange format, and forwards them to the care provider over the wide-area network. The engineering problem is therefore not a single sensor but a chain of linked requirements. The radar must sense respiration and heart rate through the mirror assembly itself, an acrylic panel carrying a two-way mirror film, which attenuates and distorts the very signal the system depends on. The display must be legible to users with impaired vision while requiring no input from them. The link to the home network must be secure and must survive that network's intermittent connectivity without losing readings, and the kit as a whole must run continuously and recover unattended on consumer

hardware after a power or network interruption. The data leaving the home must then be in a form that medical information systems can ingest rather than a private format. These reliability, security, and interoperability requirements are made concrete in chapter 3.

This project continues earlier graduation work and is best understood as a revision of it rather than a fresh start. The previous work established the broad shape of the system, including message-based communication between sensor modules and a central hub, a smart-mirror display, and a sensor module carrying environmental and radar sensors on a custom board [5]. It also exposed the parts of the design that did not withstand scrutiny, and those are what this thesis sets out to correct. The earlier security model used mutual authentication in name only: its certificate authority was generated at runtime, so no persistent chain of trust existed, and the sensor platform could not sign the handshake, so the authentication step proved nothing. Buffering against network or power outages was absent, so any interruption lost data. The present work changes the sensor platform to one capable of a real cryptographic handshake, replaces the runtime-generated trust model with static, factory-provisioned device identities, and adds the persistence required to lose no readings across an outage.

The thesis report is structured as follows. Chapter 2 presents the background research and the design space, covering the smart-mirror display and its software, radar-based presence and vital-sign sensing together with the selection of sensor modules, the communication protocols and the security model [6], the system architecture, and the integration with clinical information technology, and it closes with the intended direction for future face-recognition work. Chapter 3 states the program of requirements for the system, the sensor module, and the display. Chapter 4 documents the implementation as built, from the system overview and hardware selection through the user interface, the communication and security layer, the deployment automation, the mirror construction, and the printed-circuit-board design and radar filtering. Chapter 5 reports the results and their discussion, including radar performance through the mirror materials and the validation of the board. Chapter 6 concludes and identifies the work that remains.

1.1. Use of GenAI

Generative AI tools were used during this project, and that use is disclosed here in line with the faculty's requirements. We mainly used Claude and ChatGPT. They were used for exploring and structuring ideas, for drafting and language editing of report text, for writing and debugging code, and for assistance in locating relevant literature. Factual claims, figures, and design decisions in this report were made or checked by the authors against primary sources, and each entry in the bibliography was independently verified for existence and accuracy, because generative tools are known to produce plausible but fabricated citations. Generative AI was not used to produce measurement data or experimental results. The authors take full responsibility for the content of this report.

2

Background Research and Design Choices

2.1. Smart Mirror

A "smart mirror" is a device consisting of a display behind a two-way mirror. When the display is off, the mirror reflects light from the environment and the device functions as a normal mirror. When the display is turned on and sufficiently bright, its light passes through the mirror and the user can see both the display as well as his/her reflection.

2.1.1. Interface Design for Elderly Individuals

The display should be tailored to elderly users, and must therefore take into account any difficulties older adults encounter when using digital displays. [7] contains an overview of the ways in which older adults might experience difficulty with technology as well as design guidelines. Most difficulties with user interaction, related to motor control for example, can be avoided by requiring little to no input from the user. When designing a display however, one needs to take into account what might hinder elderly users from receiving visual information.

Older adults might experience difficulty seeing small details, and can have blank spots in the center of their vision as well as a narrowing of peripheral vision. It is therefore important to use large fonts, larger than 5mm for a computer, and to position information centrally. The use of plain fonts is recommended with special mention of Tiresias, a font specifically designed for people with visual impairment. As people age they are also less able to pick up light, detect (color) contrast, have more trouble seeing through glare and are slower to adapt to changes in brightness. Due to these factors and due to possible changes in cognition, a person might need longer to process and search for visual information, detect subtle visual design, and can be more sensitive to visual distractions. It is therefore recommended to keep the design and the information minimal and label information clearly [7].

All this needs to be considered when designing a visual display for elderly users.

2.1.2. Available Software for a Smart Mirror GUI

The smart mirror GUI could be designed from scratch and optimized for a specific use case, which would be time-consuming. There are several options available to simplify this process. LVGL (Light and Versatile Graphics Library) is a C-based open source UI framework, which is small enough to run on a microcontroller [8]. It is sufficient for designing a simple interface with minimal components. Other options exist however, which are built specifically for a wall display application and contain ready-to-go modules. The first example is DAKboard, which offers a display and can integrate with for example calendars, home assistants and weather applications [9]. It is easy to use, but limited in terms of customization and creating new modules which might require a subscription.

Another option is the well-maintained open source project called MagicMirror² [10]. It offers several in-

stallable modules as well as an API to develop new ones, making it highly customizable. MagicMirror² is developed for Raspberry Pi specifically, but can be used in other Linux environments. The MagicMirror² platform was chosen for this project because of its built-in modules combined with its customizability.

2.1.3. Smart Mirror Architecture Options

Using the MagicMirror² platform, the smart mirror can be designed in several ways. Figure 2.1 shows an overview of the configurations considered for this project.

The MagicMirror² platform can be used in two ways [10]. The first option is to use a standalone version on an SBC, and connect a display (LCD-panel, monitor, tv etc) with an HDMI cable. The second option is to run MagicMirror² in "server mode" on an SBC, and use a separate client device for the display.

This gives the following options for the smart mirror as part of the health monitoring system. Option A is to use an SBC on the sensor module, which runs a standalone version of MagicMirror² as well as sensor data acquisition. A simple display such as an LCD-panel can then be connected directly to the SBC. This option is easiest to implement, especially using a Raspberry Pi SBC, as it is what MagicMirror² was designed and tested for. The use of a simple LCD-panel and only an SBC (no separate MCU) lowers the cost.

Option B is similar, but uses the SBC only to host a server and a client device, such as a tablet, for the display. Using a tablet broadens the possibilities for the smart mirror; the integrated microphone and camera could for example be used to set up video conferences for elderly users. For a stand-alone smart mirror, either option A or B would be best depending on your desired application. However, since the mirror is part of a larger health monitoring system with multiple near identical sensor modules, every sensor module would have the same SBC. This is unnecessary and costly, as most sensor modules have no user interface attached and only handle sensor data acquisition which can be implemented with an MCU.

On the contrary, options C-E use the sensor module only for data acquisition, with the MagicMirror² platform deployed on other devices. Option C consists of a separate SBC that runs the MagicMirror² system with an HDMI connection to a display (similar to A), whereas option D uses the SBC as a host and a device such as a tablet as the client (similar to B). A drawback of this design is that it requires an additional component (both an MCU and an SBC), increasing the cost. Given that the complete health monitoring system already contains an SBC as a data hub and connection between the sensor modules and the database, it could double as the host for the MagicMirror² system. A drawback is that MagicMirror² could impact the performance of the data hub. In addition, the time available to work with the data-hub SBC for this project is limited.

Option E removes the SBC entirely, using the tablet both as the MagicMirror² host by running Linux on a virtual machine, and as its client. This is generally the cheapest option, and keeps the door open for added functionality in future by using the tablet's other components (microphone, camera etc). It is the setup used for this project.

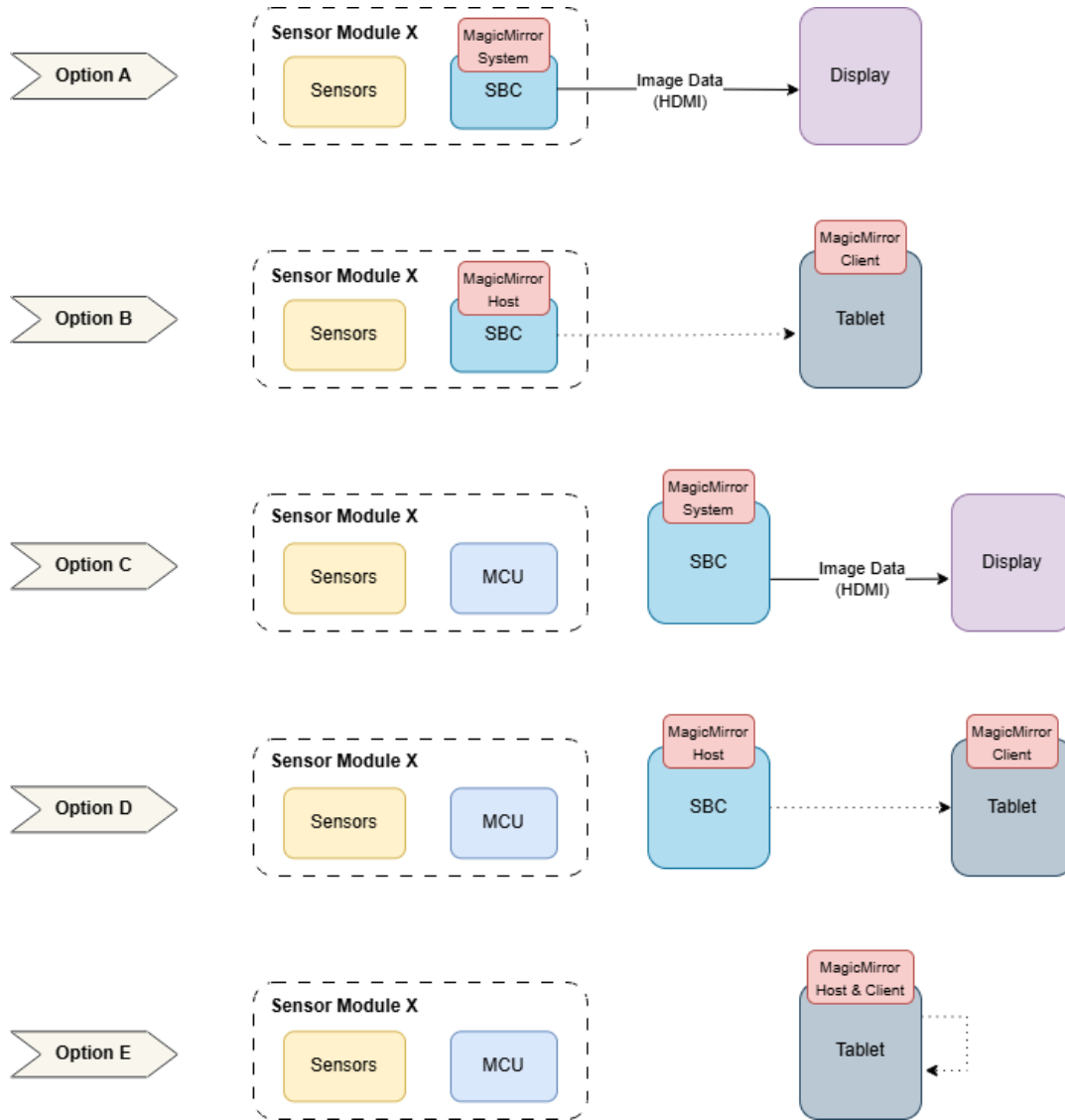


Figure 2.1: Diagram of the different architecture options for the smart mirror, as described in 2.1.3

2.2. Vital Sign Detection Using Radar Sensors

2.2.1. Background and Relevance of mmWave Vital Sign Detection

Non-contact vital sign monitoring has gained significant attention due to the demand for unobtrusive, long-term health observation [11]. MmWave radar estimates respiration and heart rate by detecting small body-surface and chest-wall displacements that modulate the reflected radar signal [11]. In particular, the short wavelength of mmWave enables the detection of millimeter-scale chest motion for respiration and heart rate monitoring [11]. With the global remote patient monitoring devices market expected to grow from USD 25.32 billion in 2020 to over USD 101 billion by 2028 [11], radar-based sensors are emerging as promising tools for home healthcare, automotive safety and emergency rescue [11].

2.2.2. Principles of mmWave Vital Sign Sensing

Radar technology transmits electromagnetic waves and measures the reflected signals from the body. Tiny chest movements caused by breathing and heartbeat modulate the phase of the reflected signal, and by processing these phase changes the system can estimate physiological parameters. Several classes of radar sensors are reported in the literature [12]: continuous-wave (CW) Doppler radars, ultra-wideband (UWB) impulse radars, frequency-modulated continuous-wave (FMCW) radars and pulsed coherent radars (PCR).

CW Doppler radars are simple but susceptible to clutter and lack distance information, resulting in poor accuracy [12]. UWB radars transmit very short pulses, provide distance information and achieve high signal-to-noise ratio, yet they require complex electronics and have high power consumption, making them costly for consumer devices [12]. FMCW radars transmit chirp signals and analyse phase changes in the reflected signal to estimate distance and detect micro-movements. They provide a good compromise between accuracy and hardware complexity, which has led to widespread adoption in vital sign detection [12], [13]. Finally, PCR sensors such as the Acconeer A121 emit ultra-short pulses and coherently receive echoes, enabling sub-millimeter distance resolution and the ability to sense minute displacements [14]. These sensors are highly integrated and low-power but typically output raw data that requires considerable signal processing to extract vital signs.

2.2.3. State of the Art

Research comparing radar technologies demonstrates the trade-offs between these classes. Wang et al. compared an IR-UWB radar and an FMCW radar for vital sign monitoring and found that the UWB radar achieved higher signal-to-noise ratio and stronger correlation with a reference sensor. Pearson correlation coefficients were 0.95 for respiration rate and 0.97 for heart rate using IR-UWB, compared with 0.88 and 0.91 respectively for FMCW radar [15]. However, the FMCW radar uses simpler hardware and is therefore less expensive, which is advantageous for cost-sensitive applications [15].

Another study evaluated a 77-GHz FMCW radar and reported detection accuracies greater than 93% for both heartbeat and respiration rate [13], demonstrating the feasibility of FMCW radar for high-accuracy, contactless monitoring. More recent work combines FMCW radar with advanced signal processing such as wavelet decomposition and adaptive Kalman filtering and achieved estimation error rates below 7% under different distances and postures [12]; while promising, such approaches increase algorithmic complexity and development time.

In addition to research prototypes, commercial radar modules are available. The Acconeer A121 module uses 60 GHz pulsed coherent radar technology and can detect presence, motion and vital signs by analysing phase changes in the echoes [14]. Its highly integrated design offers low power consumption and configurable detection zones, but because it outputs raw radar data it requires special signal processing algorithms to retrieve respiration and heart rate information. This increases integration effort and is not ideal for rapid prototyping.

2.2.4. Commercial Radar Modules for Vital Sign Detection

Some FMCW-based modules include built-in algorithms for vital sign extraction. The DFRobot C1001 is a 60 GHz mmWave sensor marketed primarily for fall and presence detection but includes a breathing and heart-rate mode. According to the manufacturer's tests, the sensor can detect breathing rates between 10 and 25 breaths per minute and heart rates between 60 and 100 bpm within 0.4–1.5 m of the chest [16]. The module can detect motion up to 11 m [16] and presence up to 4 m [16]. However, it is intended for general-purpose monitoring rather than medical use. The manufacturer warns that it is not a professional medical device and should not be used as an auxiliary accessory for diagnosis [16]. Heart-rate measurements show some variance and require careful sensor placement [16].

The MR60BHA2 module from Seeed Studio is specifically designed for breathing and heart-rate monitoring. It is based on a 60 GHz FMCW radar and includes on-board signal processing. According to the technical specification, the module achieves contactless breathing and heartbeat detection with a range of 0.4–1.5 m and claims measurement accuracies around 90% [17]. The module operates over a wide temperature range and provides multiple communication interfaces, including UART, I²C and SPI, allowing access to raw radar data for further development [17]. A review compared the MR60BHA2 outputs to a Garmin watch and manual counting and found that heart-rate readings differed by only 1–2% and respiratory rates matched closely [4], which increase confidence in the reliability of the integrated algorithms. The module also supports static presence detection up to around 6 m [4]. At approximately 22 euro, it offers an affordable, dedicated solution for vital sign detection.

Given the constraints of our project, developing a concept demonstrator of a magic mirror that displays heart rate and respiratory rate to a user, the time available does not allow for implementing complex signal-processing algorithms from raw radar data. While research radars, such as 77-GHz FMCW or 60-GHz PCR, offer high accuracy, they require very complex algorithms and substantial development

effort. Commercial modules that output vital sign information directly are therefore preferable. Comparing the C1001 and MR60BHA2 modules, the C1001 is a multi-purpose fall and presence detector with limited accuracy and no medical certification [16], whereas the MR60BHA2 is purpose-built for breathing and heartbeat monitoring, with higher claimed accuracy [4]. Despite the C1001 offering a longer presence detection range and lower power consumption, its accuracy and reliability for vital sign measurements raise concerns for our objective. Consequently, the MR60BHA2 module is recommended as the vital signs radar sensor for our application. Although the MR60BHA2 is more suitable for this demonstrator because it is designed for breathing and heartbeat monitoring and outputs processed values, it is still not a clinically validated medical device.

2.2.5. Comparison of mmWave Radar Sensor Modules

Table 2.1: Comparison of radar sensor modules for vital sign detection.

Specification	MR60BHA2	C1001
Vital sign detection range (m)	0.4–1.5	0.4–1.5
Measurement accuracy	≈90% [17]	Not specified; manufacturer notes proper placement is required [16]
Radar beam (degrees)	60 × 60 [17]	100 × 100
Communication interfaces	UART, I ² C, SPI, GPIO [17]	Tx, Rx, IO1, IO2
Operating temperature (°C)	-20 to 85 [17]	-20 to 60 [16]
Intended application	Breathing and heartbeat monitoring [17]	Fall detection and presence; basic breathing/-heartbeat mode
Power consumption (mA)	≈600 [17]	≈100 [16]
Static presence detection range (m)	≈6 [4]	11 [18]
Approx. price (euro)	≈22 [19]	≈25 [18]

2.2.6. Presence Detection Radar Sensor Selection

In addition to the vital sign radar module, a separate presence radar sensor is considered.

Table 2.2: Comparison of commercial presence radar sensor modules.

Sensor module	Detection range	Approx. price (euro)	Evaluation
Waveshare HMMD mmWave Sensor	Moving: 10 m / micro-motion: 6 m [20]	2.56 [20]	Best choice: cheap, compact, long range, and designed specifically for presence detection.
Hi-Link HLK-LD2410	Up to 5 m [21]	5.25 [21]	Good alternative; slightly more expensive but features a shorter maximum range than the HMMD.
DFRobot C4001 24 GHz	Up to 12 m [22]	11 [22]	Better range detection but more expensive.

The HMMD is a 24 GHz FMCW module optimized for presence detection and micro-motion sensing. It detects moving targets at 10 m and even subtle micro-motions at 6 m [20]. Additionally, compared to other commercially available modules, the HMMD is the cheapest while still presenting the needed performance for our application. The price factor is very important for our application. The HMMD sensor is available for 2.56 euro [20]. This price can be much lower for mass production. Also, the shipping cost for 1000 pieces, about 2 kg, is about 35 when choosing the UPS shipping method.

Therefore, the HMMD module is recommended as the presence detection sensor. It provides sufficient range and it is designed for human presence and micro-motion detection, and has the lowest cost among the compared modules.

2.3. Communication Protocols and API Integration

The previous BAP [5] established MQTT over mutual TLS (mTLS) between sensor modules and a central hub, with UDP broadcast for broker discovery and NTP for clock synchronisation, using `umqtt.simple` on MicroPython sensors and Mosquitto on the single-board computer (SBC), with JSON payloads (~94 bytes) and a validated round-trip latency of ~84–98 ms. The hub printed measurements to a terminal; the leg toward external medical infrastructure was not implemented. The foundational protocol references inherited from the previous theses (MQTT-SN, the Naik comparison [6], TLS) remain valid for background and are not repeated here. This section argues the design and the changes from the previous work; the concrete configuration, parameters, and scripts are given in the implementation chapter (Sections 4.3 and 4.4).

The present revision keeps the protocol family and replaces the parts of the previous design that did not survive scrutiny: the security model, the broker placement, the discovery mechanism, the persistence story, and the previously-unimplemented external leg. The sensor platform also changes, from an ESP32-S3 running MicroPython to a Raspberry Pi Pico 2 W (RP2350) running C against the Pico SDK, lwIP, and mbedTLS, which is what makes a real mTLS implementation possible.

2.3.1. Protocols

Recent comparative work [23] confirms what Naik [6] established: MQTT is stable under intermittent connectivity at modest CPU cost; CoAP is more resource-efficient but loses MQTT's broker-mediated features (retained messages, last-will, session persistence); HTTP degrades sharply under load. For five to ten sensor modules per home on residential Wi-Fi, MQTT is the right operating point, and the revision happens at configuration and provisioning rather than by switching protocol family. Table 2.3 summarises the criteria.

Table 2.3: Application-protocol comparison against criteria relevant to the residential health-monitoring deployment.

Criterion	MQTT	CoAP	AMQP	HTTP/REST
Transport	TCP	UDP	TCP	TCP
Pattern	Pub-sub	Req-resp	Pub-sub	Req-resp
Header overhead	Low (~2 B)	Very low	Higher	Highest
Reliability options	QoS 0/1/2	Confirmable	Persistent	TCP-only
Broker required	Yes	No	Yes	No
Behaviour under load	Stable	Stable (UDP)	Stable	Degrades sharply [23]

2.3.2. Security: Truly Static mTLS

The previous mTLS implementation [5] was neither truly static nor cryptographically real. The certificate authority was generated on the SBC at runtime, so there was no persistent chain of trust; the sensor's private key was loaded with the firmware but could not be used for signing because of MicroPython's TLS limitations; and the registration nonce was returned to the hub unsigned, so the handshake proved nothing. The previous thesis itself acknowledges that the mTLS should ultimately be static and issued by a real authority, which the present revision implements.

The platform shift to the Raspberry Pi Pico 2 WH is what enables this. The Pico SDK exposes mbedTLS, so the sensor performs a genuine ECDSA P-256 mutual-authentication handshake in which it signs the handshake transcript with its private key. Because the TLS handshake already proves possession of the private key, the previous design's separate signed-nonce step is redundant; it was a homegrown reimplement of what mutual TLS provides natively. The revision therefore removes the registration handshake entirely: there is no enrolment phase, no nonce challenge, and no bootstrap credential. Each device is born with its identity.

That identity is a static, factory-provisioned certificate. The ECDSA P-256 keypair is generated off-device at a controlled provisioning station, a certificate-signing request with the device UUID as its common name is signed by a project certificate authority, and the signed certificate, private key, and CA certificate are written to the device's flash before deployment. The same authority issues the certificates for the other parties on the local network as well, the smart mirror, the operator role, and the gateway, so the whole local-area leg shares one trust anchor. The device never generates keys, never produces a CSR, and never holds the CA private key; the CA private key never touches a deployed device or the source repository, which directly addresses the previous design's leak of an unencrypted CA key into a public repository. This is the static, manufacturer-installed end of the device-identity spectrum, analogous to the IDevID model that automated-bootstrap protocols build on. Automated onboarding was considered and rejected: Demchenko et al. [24] introduced the zero-touch model, Boskov et al. [25] applied it to IoT with time-to-provision data, and the IETF standardised it as BRSKI in RFC 8995 [26], but its onboarding machinery is unnecessary complexity for a small residential deployment. Its operational guidance is adopted, however: RFC 8995 endorses re-provisioning over online revocation (OCSP) for constrained devices, and the revision follows this, with no on-device certificate rotation and re-provisioning as the response to compromise.

One limitation is stated plainly rather than hidden. In the present version the device key resides in flash unencrypted and is therefore exposed to an attacker with physical flash access; the RP2350's one-time-programmable memory and encrypted-flash regions are identified as the hardening path for a later iteration. This is an accepted trade-off for the project timeline, not an oversight.

Performance evidence supports the platform. Tschofenig and Pegourie-Gonnard [27] benchmark mbedTLS on a Cortex-M4 with ECDSA P-256 handshakes around 0.4 s; the RP2350's Cortex-M33 with a hardware true-random generator and SHA acceleration sits above that class, so the handshake cost is comfortably acceptable. Gavrilidis et al. [28] show TLS-enhanced MQTT is feasible on Raspberry Pi 4B-class hardware, and Rzepka et al. [29] provide the cross-platform Pi-versus-MCU comparison. Crucially, the TLS cost is a one-time handshake, not a per-message tax, so the steady-state latency is expected to remain comparable to the previous plaintext baseline. The cipher suite is restricted to ECDHE-ECDSA with AES-GCM, excluding CBC, RSA, and SHA-1. TLS 1.2 is the floor and 1.3 the target; the deployed handshake is currently 1.2 for toolchain reasons detailed in Section 4.3.

2.3.3. Simplicity: Keep the Existing Stack

Ecosystem maturity directly addresses the timeline constraint. Eclipse Mosquitto and the Paho client family are mature and well-documented; Tsao et al. [30] demonstrate Mosquitto-based vital-signs monitoring across five medical-device types. The simplicity-positive choice is to keep this stack and revise the configuration around it: mDNS for discovery, lwIP's built-in MQTT client on the sensor (the Pico runs C, not CPython, and the Paho client lives only on the gateway subscriber), and certificate-derived access-control for per-device scoping.

The same reasoning fixes the wire format. JSON (RFC 8259) is retained end to end, for both the MQTT payloads and the on-device configuration, after a brief detour toward CBOR was reversed: FHIR R4 is JSON-native and the smart-mirror UI consumes JSON, so a single serialiser and parser cover the wire format, command parsing, and on-flash state, and the marginal byte savings of CBOR are irrelevant at this system's aggregate rate of roughly one kilobyte per second. Per-device authorisation is enforced by the broker without any per-device configuration, by keying its access-control rules on the certificate common name so that each device is confined to its own topic subtree; the pattern syntax that achieves this is given in Section 4.3.

2.3.4. Summary of Changes

Below the changes 2.4 made to the communication part of the project relative to the previous group.

Table 2.4: Communication changes between the previous design [5] and the present revision.

Area	Previous	Revised
Sensor platform	ESP32-S3 + MicroPython	RP2350 (Pico 2W) + C, lwIP MQTT + mbedTLS
Broker location	On the SBC	On the Android tablet (Termux)
Device key and signing	Pre-loaded; unusable for signing (MicroPython)	Factory-provisioned ECDSA P-256; genuine mutual-auth signing via mbedTLS
Onboarding	Enrol–verify with unsigned nonce	None; static factory-issued certificate
Certificate authority	Generated on the SBC at runtime	Project CA, offline; key never on a device or in the repository
TLS version / cipher	TLS 1.2, AES-CBC	TLS 1.2 (1.3-ready), ECDHE-ECDSA-AES-GCM
Discovery	UDP broadcast	mDNS; static-IP re-provision as fallback
Reconnection	5 fixed retries	Exponential backoff
Outage persistence	None	Device-side spool + Gateway-side store-and-forward, idempotent
Per-device authorisation	Shared bootstrap credential	Certificate-CN access-control
External API (WAN leg)	Not implemented	FHIR R4 over HTTPS (auth out of scope)

2.3.5. The Two Communication Legs

The communication design has two legs with different requirements; the tier topology that realises them, and its diagram, are given in Section 2.4. On the local-area leg the sensor modules and the gateway are all clients of one broker, speaking MQTT over mTLS at QoS 1, and the single trust anchor of Section 2.3.2 authenticates each of them. On the wide-area leg, which is new for this project, the gateway carries readings to the medical IT systems as HL7 FHIR over HTTPS. Famá, Faria and Portugal [31] describe the closest published precedent: a deployment with the same two legs, MQTT over TLS from the sensors to the broker and FHIR over HTTPS from the translator to the medical IT systems, in which one device serves as both broker and FHIR translator. The present design separates those two functions onto two devices, the tablet that hosts the broker and the single-board computer that performs the translation, which this thesis calls the gateway.

2.3.6. HL7 FHIR

HL7 FHIR R4 [32] is the international standard for exchanging healthcare data, and using it is non-negotiable for the external leg. The resources this system needs are *Observation* (one per sensor reading, carrying a coded type, value, units, timestamp, subject, and device reference), *Bundle* for grouping Observations into a single request, *Device* for the sensor module, and *Patient* for the resident, pre-provisioned by the medical IT systems. Vital-sign measurements are LOINC-coded under the FHIR Vital Signs Profile [33], whereas the ambient air-quality measurements (CO₂, TVOC, AQI) have no widely-adopted LOINC codes, which is one of the open questions below. The HL7 Personal Health Device Implementation Guide [34] targets the home-uploading-via-gateway topology used here and is the conformance reference.

The translation is the gateway’s responsibility, not the sensor’s: no FHIR resource, LOINC code, or clinical terminology runs on the microcontroller, which emits only compact sensor JSON. The schema of that JSON is designed so that every field a FHIR Observation needs is either present or derivable from context the gateway already holds, which is what keeps the clinical-terminology surface off the constrained device. Two recent precedents inform this layer: Adochiei et al. [35] build a full open-source IoT-to-FHIR stack that the gateway-side design effectively ports, and Marques et al. [36] describe a real-world fog-IoT-to-FHIR deployment with over 4,500 users in elderly and chronic-condition populations, providing proof-of-scale evidence for the target demographic. The concrete code mapping, the

deduplication and idempotency scheme, and the dead-letter handling are deferred to Section 4.3.

2.3.7. Deployment and Open Questions

Deployment is automated so that the kit recovers without manual intervention, across two self-healing surfaces: the tablet, which brings up the broker, discovery, and the mirror after a reboot, and the gateway, which brings up the aggregation and translation service. Two cross-tier loops are worth noting at the architectural level: the tablet supplies wall-clock time to the sensors, which otherwise have no real-time clock, and the gateway alone holds the device-to-patient binding, so that a sample from an unbound device is retained rather than discarded and is processed once the binding is made. The scripts that implement this are described in Section 4.4.

Five questions remain for the medical-infrastructure stakeholder before external-leg implementation: whether the medical IT systems endpoint expects FHIR R4 or R5 (R4 is the safer default); which authentication mechanism the endpoint requires, since SMART on FHIR Backend Services with OAuth 2.0 is the likely answer but is out of the current scope; whether Patient resources are pre-provisioned by the medical IT systems or must be created by the system (pre-provisioning strongly preferred); which LOINC or SNOMED codes are appropriate for ambient sensing (presence, CO₂, TVOC), since vital signs are well-coded but ambient measurements are not; and how a single home with multiple residents is disambiguated, since the present version binds one device to one patient at provisioning time and runtime resident identification is deferred to the future face-recognition work of Section 2.5.

2.4. System Architecture

The monitoring system is organised as three on-premises tiers that pass data toward an external medical IT systems endpoint, as shown in Figure 2.2. At the edge, microcontroller-based sensor modules sample radar, environmental, and light sensors and publish their readings. A module acts only as a thin publisher: it formats each reading as a compact JSON message and does nothing else with it. Every module reaches the tablet over Wi-Fi, and every message is carried by MQTT inside a mutually authenticated TLS session (mTLS), so a counterfeit module cannot inject false readings into the system.

The edge tier carries two module roles that share the same firmware and the same transport. The first is paired with the smart mirror in its room: it senses and publishes, and the mirror renders its vitals locally. The second is a generic module for rooms that have no mirror. It carries the same on-board OLED, buttons, and LEDs as the mirror-paired module and differs only in its sensing front end, and it publishes to the same broker over the same secured transport. What a generic room lacks is the two-way mirror display, not the module's own status screen. The same design therefore scales to any number of rooms: each additional module is provisioned with its own identity and publishes under its own topic namespace, with no change to the broker, the gateway, or the protocol.

The tablet fills two roles at once. It hosts the MQTT broker, the single point through which all sensor traffic passes, and it runs the smart-mirror display, which subscribes to that same broker to render the vitals behind the two-way mirror for its paired room only. Locating the broker on the tablet is a fixed constraint of the project rather than a derived design choice, and the rest of the architecture is built around it.

The gateway tier is a single-board computer that subscribes to the broker over the local network, again over MQTT with mTLS. It aggregates the raw readings from every module, translates them into the standard clinical exchange format (HL7 FHIR Release 4), and buffers each resource until the medical IT systems confirms receipt, so a dropped link loses no data. The gateway is the only tier that holds clinical terminology and the only tier that communicates outside the home: it posts FHIR resources to the healthcare provider's server over HTTPS across the wide-area network. The scope on that leg is the FHIR application interface itself, the construction and posting of valid resources to a FHIR server; authentication to a production endpoint is left to the provider's chosen mechanism (Section 2.3) and is not exercised in this work. This split keeps all heavy translation and clinical context on the capable gateway and keeps every edge module small, replaceable, and free of any patient-identifying data.

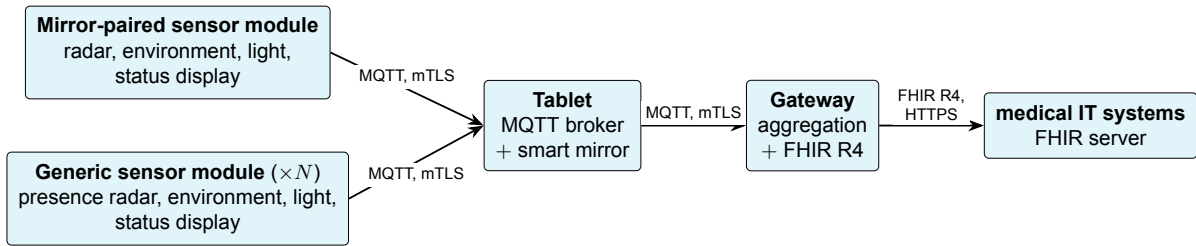


Figure 2.2: High-level system architecture: a mirror-paired module and N generic modules publish to one tablet broker, which feeds a single gateway delivering to the medical IT systems endpoint.

2.5. Future Work: Face Recognition

Face recognition in this project identifies which resident is standing in front of the smart mirror, so that the mirror can show that resident's personalised vital-sign trends, medication reminders, and appointment summaries fetched from the medical IT systems FHIR server (Section 2.3). It is closed-set authentication for a small, known set of household members, not a general-purpose biometric identifier. A wrong identification surfaces one resident's medical data to another, which is a worse failure than identifying no one, in which case the mirror falls back to generic content. The camera position, viewing angle, distance, and number of possible users are all fixed in advance, so the task is closed-set local recognition for typically one to five enrolled residents. This is future work, so what follows describes the intended direction rather than an implemented result. Two approaches are viable: a self-contained vision module, which the project supervisor suggested, and a software pipeline running on the single-board computer (SBC). Both are presented below, together with the criterion that would choose between them; they share the same host-side decision logic and the same privacy posture, and differ only in where the recognition runs.

2.5.1. A Self-Contained Vision Module: HuskyLens

The DFRobot HuskyLens is a self-contained machine-vision sensor built around the Kendryte K210, a dual-core RISC-V system-on-chip with a dedicated neural-network accelerator. It performs the entire vision pipeline on the device: a built-in face-recognition function detects a face, matches it against faces it has previously learned, and assigns each learned face an identity. Enrolment is a physical step, a learning button that captures a face, optionally from several angles to improve recognition across viewpoints, and binds it to an identity; the same learning action can also be triggered programmatically. The module reports its result, the recognised identities and their image coordinates, to a host over UART (configurable up to one megabaud) or I2C, so the host receives an identity rather than an image and never has to run a recognition algorithm itself [37].

For future work this is an efficient fit. There is no custom model training, no model-conversion toolchain, and no inference workload on the SBC; the vision computation lives entirely on the module, which removes the compute contention with the protocol and FHIR stack that an on-SBC pipeline would introduce. Integration rides the architecture already in place: the module is wired over UART or I2C to a host at the mirror, and that host republishes the recognised resident identity onto the existing mutual-TLS MQTT fabric (Section 2.3), where the mirror consumes it to personalise its display and the SBC consumes it to select the corresponding FHIR Patient. Figure 2.3 shows the flow.

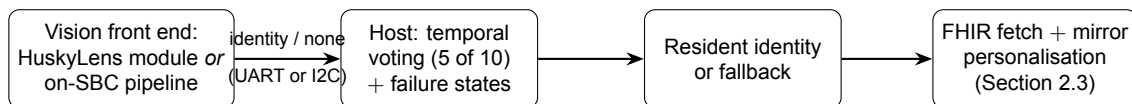


Figure 2.3: Future face-recognition flow. The vision front end, either the HuskyLens module or an on-SBC BlazeFace and MobileFaceNet pipeline, emits a per-frame identity. The host applies temporal voting and explicit failure states before the identity drives the FHIR fetch and the mirror personalisation of Section 2.3.

2.5.2. Decision Logic and Failure States

Because a misidentification is the costly failure, the host must not commit to an identity from a single frame. Each per-frame result from the vision front end is treated as a vote, and an identity is emitted only when at least five of the last ten frames agree (Figure 2.4). This temporal voting trades a small amount of latency for a lower false-accept rate, which is acceptable because the mirror displays information rather than reacting instantaneously. The host also enforces explicit failure states, no face, multiple faces, and an unrecognised face, so that the system never silently emits a wrong identity; on any of these the mirror reverts to generic content. This host-side logic, not the vision front end, is where the privacy requirement is actually enforced.

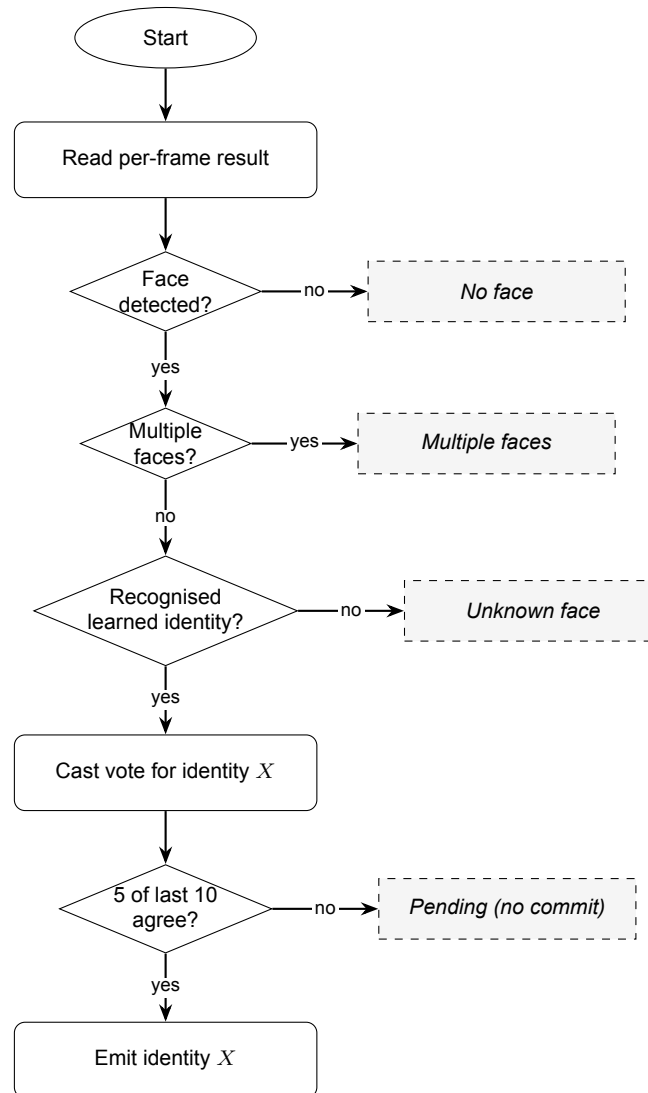


Figure 2.4: Recognition decision logic with temporal voting. Each frame either reaches the voting stage or short-circuits to one of three explicit failure states. An identity is emitted only when at least five of the last ten frames agree; every failure state falls back to generic mirror content.

2.5.3. The Two Approaches Compared

The two approaches trade different things. The HuskyLens module is the faster path: no model training, no conversion toolchain, no inference load on the SBC, and a strong data-locality property in that raw frames never leave the module. Its cost is that the on-device face model is a closed component that cannot be retrained, audited, or have its similarity threshold tuned through the standard interface, its recognition accuracy across lighting and pose is not characterised in any published figure, and its neural accelerator is far smaller than the SBC's, by more than an order of magnitude.

The on-SBC software pipeline is the more rigorous path: a lightweight detector such as BlazeFace [38] feeding a compact embedding model designed for embedded inference such as MobileFaceNet [39], with cosine matching against locally stored reference embeddings. It costs the effort of building and validating the pipeline and it consumes SBC compute, but it returns a tunable decision threshold and a model that can be audited and replaced. Both approaches rest on the same principle, that a face maps to a numerical embedding in which same-person faces lie close together, established by FaceNet [40] and refined by ArcFace's additive angular-margin loss [41]; HuskyLens packages a small such model behind a fixed interface, whereas the SBC pipeline keeps it open. From the rest of the system's point of view the two are interchangeable, because both emit a per-frame identity into the same host-side decision logic (Section 2.5.2).

The selection criterion is accuracy under the privacy requirement. Because the cost of a false accept is disclosure of protected health information, whichever approach is taken must first be measured for recognition accuracy and false-accept rate against the actual enrolled set. The module is the natural choice for a first feasibility build, given its speed and its data locality; the SBC pipeline is the natural choice if that measurement shows the module's fixed model or untunable threshold cannot meet the required false-accept rate.

2.5.4. Privacy and Hardware

Local processing is the privacy baseline, and the module approach meets it strongly: raw frames are processed and discarded on the module, the learned model and reference data stay on the module (optionally saved to its own removable storage), and only an integer identity crosses to the host, so no image or embedding leaves the camera. The caveat is that this on-module store is outside the system's own audit control, which is part of why measured accuracy, not just data locality, gates any deployment.

Under the module approach the vision workload does not run on the SBC, so it does not contend with the protocol and FHIR stack for compute; under the on-SBC approach it does, which is one of the trade-offs above. Either way the SBC is selected on system grounds rather than on face recognition, and its neural accelerator leaves headroom for the on-SBC pipeline and simply goes unused by face recognition if the module is chosen.

Two approaches are viable for future face recognition, and both fit the rest of the system unchanged because both emit a per-frame resident identity into the same host-side logic. The first is the DFRobot HuskyLens (Kendryte K210), a self-contained module that performs detection, embedding, and matching on-device and emits only an identity over UART or I2C; it is the faster path and keeps raw images on the module. The second is an on-SBC pipeline (BlazeFace with MobileFaceNet and cosine matching), which is slower to build but gives a tunable threshold and an auditable, replaceable model, and for which the Dragon Q6A's neural accelerator is available. In both cases a host republishes the identity onto the existing mutual-TLS MQTT fabric, where the mirror personalises its display and the SBC selects the corresponding FHIR Patient, and in both cases the host applies temporal voting over five of the last ten frames with explicit failure states so that a wrong identity is never emitted. The choice between them is gated on one measurement, recognition accuracy and false-accept rate against the actual enrolled set, since a false accept discloses protected health information. The module is the natural first build; the SBC pipeline is the answer if that measurement falls short.

3

Program of Requirements

The objective of this project is to develop a remote health monitoring system for elderly clients living independently at home, built around a smart mirror that serves as the system's local hub and interface. Placed in the living room, the mirror integrates a sensor module measuring vital signs and environmental data, an MQTT message broker, and a user interface shown through the mirror surface. Additional sensor modules are distributed across other rooms, each publishing measurements to the broker; the modules are modular, with their component set depending on placement. A single board computer acts as the aggregation gateway between the home and the care provider: it subscribes to the broker, collects the measurements from all modules, and translates them into LOINC-coded HL7 FHIR R4 Observation resources posted over HTTPS, bridging local MQTT to outward-facing HTTPS as the only component that faces the external network.

3.1. Smart Mirror Requirements

1. The smart mirror must contain the sensor module
2. The system should detect the heart rate and respiratory rate of a person standing in front of the mirror
3. The smart mirror should send data to a Single Board Computer which operates as a gateway between the smart mirror and a health care server

3.1.1. Sensor Module

1. The following components should be part of the sensor module.
 - Either the ENS160 + AHT21 or the BMP280 environmental sensor, using a modular design
 - GL5516 light sensor
 - Radar sensors: C1001 or MR60BHA2 or HMMD in a modular design
 - Status LEDs for internet connection and power
 - Status display
 - MCU
 - Buttons to control the small display
 - Reset button
2. The sensor module must use radar sensors to measure heart rate and respiratory rate
3. The sensor modules must use radar sensors to detect presence

3.1.2. User Interface

1. The system must not require input from elderly users.

2. The display must turn on automatically when a person stands in front of the mirror.
3. The display must turn off automatically after a person has moved away from the mirror.
4. The display must represent vital signs and environmental data
5. The display should present data in an easily understandable way for elderly users

3.1.3. Sensor Module Display

- The sensor module display should turn on when a button is pressed.
- The sensor module display should turn off automatically after a given time if no button is pressed
- The buttons of the sensor module should be able to control the information visible on the screen
- The sensor module display should depict the following:
 - WiFi status and its strength in dB's
 - Location of sensor module within the home
 - Current time and data
 - Environmental measurements
 - Heart and breath rate measurements
 - Sensor ID's of environmental and UWB sensors

3.1.4. Communication Between the Sensor Module and the Broker

1. The sensor module publishes all measurements over MQTT to the broker, secured with mutual TLS (mTLS), TLS 1.2 minimum and TLS 1.3 where available, using ECDSA P-256 certificates. The single-board computer (SBC) consumes these measurements as a separate subscriber to the same broker; the module and the SBC never exchange data directly.
2. Each device authenticates with its own unique, factory-provisioned X.509 certificate. There are no shared passwords, no shared keys, and no runtime enrollment step. A device's identity is fixed at provisioning time.
3. No measurement is lost during a Wi-Fi or broker outage. The sensor module persists at least 15 minutes of measurements to local non-volatile flash and, once the link is restored, resends them in their original order with their original timestamps. Delivery uses MQTT QoS 1 (at-least-once), retried until the broker acknowledges each message.

3.1.5. Integration with Existing Health Care IT Infrastructure

1. Vital-sign measurements are exposed as HL7 FHIR Release 4 Observation resources.
2. Each Observation carries a standard LOINC code: heart rate 8867-4 and respiratory rate 9279-1. Environmental measurements (ambient temperature, humidity, air quality) use a documented mapping to LOINC where a suitable code exists and otherwise to SNOMED CT or a documented custom URN.
3. All outbound traffic to the medical or cloud backend uses HTTPS with mutual TLS (TLS 1.2 or newer).
4. If the medical or cloud endpoint is unreachable, the SBC queues outbound Observations locally for at least 24 hours and replays them idempotently when the endpoint returns. Each replayed Observation carries an Observation.identifier so that the receiving system can deduplicate it.

3.1.6. Deployment Script

1. A single deployment script brings the gateway from a clean device to a fully running system, with no manual configuration steps, so that an installation is reproducible.
2. All deployment settings (broker address, FHIR endpoint URL, site identifier, and network credentials) live in one external configuration file, kept separate from the application code.
3. After a power cycle the gateway boots unattended, with no manual intervention, and is operational within 60 seconds. The gateway is defined as operational once it has reconnected to the broker,

re-subscribed to the device topics, and resumed translating and forwarding (or, if the endpoint is unreachable, queuing) Observations.

4. If the gateway aggregation and delivery service stops unexpectedly, it is restarted automatically within 10 seconds without a full reboot.

4

Implementation

4.1. System Overview

Section 2.4 introduced the system as three on-premises tiers feeding an external medical IT system end-point, with two edge roles sharing one firmware: a mirror-paired sensor module and a generic sensor module that scales to the other rooms. This section gives the concrete realisation of that architecture and the hardware chosen for each tier; the per-subsystem detail follows in the remaining sections of this chapter, and the configuration and parameters are deferred to Sections 4.3 and 4.4. Table 4.1 lists the selected components for both module variants and the shared tiers, Figure 4.1 shows how they connect in the as-built system, and the data path that links them is described at the end of the section.

Table 4.1: Hardware selected for the as-built system.

Element	Selected part	Interface and function
<i>Sensor module, mirror-paired variant</i>		
Microcontroller	Raspberry Pi Pico 2 WH (RP2350)	CYW43439 Wi-Fi; firmware host
Environmental sensor	Aosong AHT21	I ² C; temperature, humidity
Air-quality sensor	ScioSense ENS160	I ² C; eCO ₂ , TVOC, air-quality index
Vital-sign radar	Seeed Studio MR60BHA2 (60 GHz)	UART; presence, heart rate, breath rate
Ambient-light sensor	Rohm BH1750	I ² C; illuminance
Status display	2.08-inch 256×64 OLED (SH1122)	I ² C; on-device status pages
User input	Two push buttons, status LEDs	GPIO; local interaction
<i>Sensor module, generic variant (rooms without a mirror, ×N)</i>		
Microcontroller	Raspberry Pi Pico 2 WH (RP2350)	CYW43439 Wi-Fi; firmware host
Presence radar	WaveShare HMMD (24 GHz)	UART; presence and movement only
Environmental sensor	Bosch BMP280	I ² C; temperature, pressure
Ambient-light sensor	GL5516 photoresistor (LDR)	ADC; room-light on/off for activity tracking
Status display	2.08-inch 256×64 OLED (SH1122)	I ² C; on-device status pages
User input	Two push buttons, status LEDs	GPIO; reset and OLED page switch
<i>Tablet</i>		
Tablet	ITMEIPC IT-701A (Android, Termux)	Wi-Fi; MQTT broker and mirror display
<i>Gateway</i>		
Single-board computer	RockPro64 (demonstrator); Radxa Dragon Q6A (target)	MQTT subscriber; FHIR R4 translator and store-and-forward

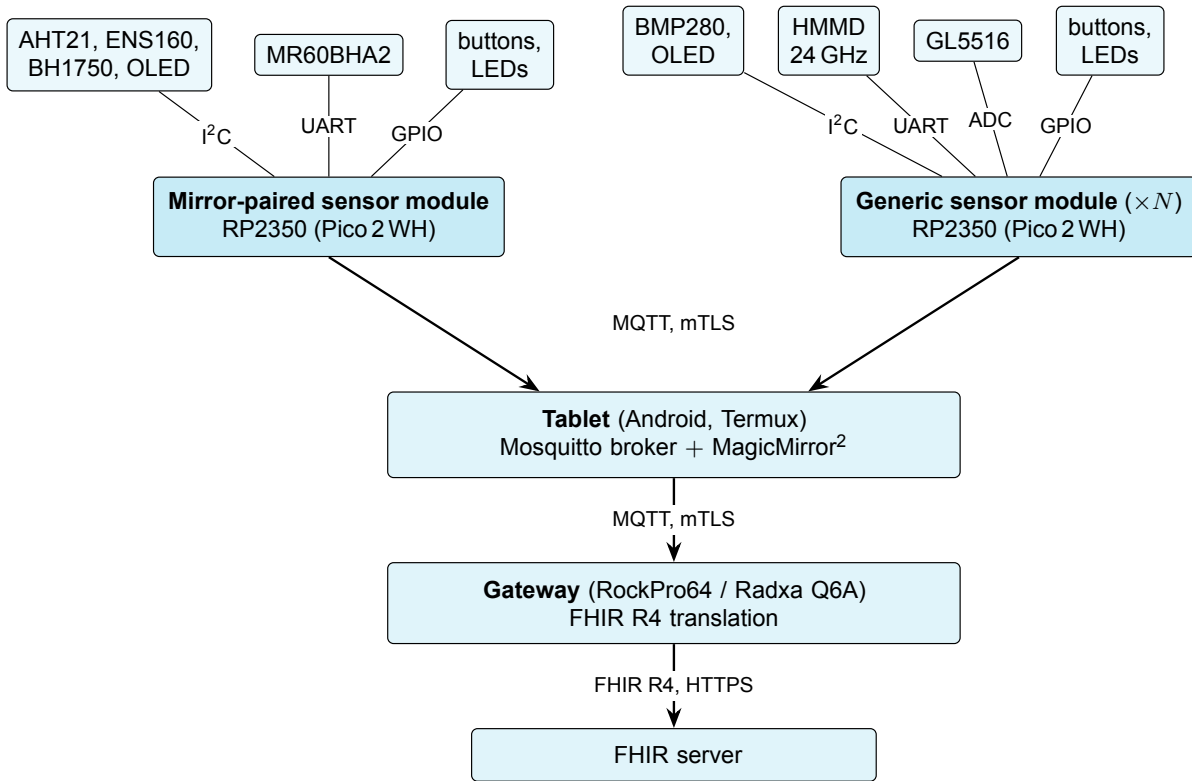


Figure 4.1: As-built system. Each module wires its sensors and interface to an RP2350 over I²C, UART, ADC, and GPIO; both publish to the tablet broker over MQTT with mTLS, and the gateway translates to FHIR R4 for the medical server.

4.1.1. Mirror-paired Sensor Module

This module carries the full sensor complement; the generic module follows. It is built around a Raspberry Pi Pico 2 WH, whose RP2350 microcontroller (dual Cortex-M33, 520 KB SRAM, 4 MB flash) and CYW43439 radio run the firmware and carry the Wi-Fi uplink. Choosing this part over the previous ESP32-S3 is what makes the mbedTLS-based mutual TLS of section 4.3 possible, and the radar selection is argued in chapter 2. Four devices share a single I²C bus: an Aosong AHT21 for temperature and humidity, a ScioSense ENS160 for equivalent CO₂, total volatile organic compounds, and an air-quality index, a Rohm BH1750 for illuminance, and a 2.08-inch 256×64 OLED driven by an SH1122 controller that shows on-device status pages. The Seeed Studio MR60BHA2 60 GHz radar, which reports presence, heart rate, and breath rate, is read over a UART rather than the shared bus. Each sensor sits behind a driver abstraction, so the build can substitute parts without code changes: in this demonstrator the environmental sensor is the AHT21 and the light sensor is the BH1750, with a GL5516 photoresistor on an analogue input as the alternative light-sensor populate option. Two push buttons and status LEDs complete the local interface. The module is always mains-powered, with no battery and no sleep mode: a 5 V input feeds the microcontroller, whose 3.3 V rail powers the bus devices, display, and inputs, while the radar runs from its own 5 V rail to isolate its radio-frequency draw. No image sensor is present on the module; any future face recognition is handled off-module (Section 2.5). The firmware is written in C against the Pico SDK with FreeRTOS, lwIP, and mbedTLS, and it acts only as a thin publisher: each sensor is sampled by its own task, and a single transport task formats every reading as compact JSON and publishes it. The board layout is covered in subsection 4.6.5, the radar plausibility filtering in section 4.8, and the messaging in section 4.3.

4.1.2. Generic Sensor Module

Rooms without a mirror use a generic variant of the same module. It runs the same RP2350 firmware on the same board design and keeps the same on-board interface: the 2.08-inch OLED on the shared I²C bus, two push buttons (one for reset, one to switch the OLED page), and the status LEDs on GPIO.

It differs from the mirror-paired variant in its sensing front end: the presence radar is a WaveShare HMMD 24 GHz module read over a UART, the environmental sensor is a Bosch BMP280 on the same I²C bus, and the light sensor is a GL5516 photoresistor on an analogue input rather than the digital BH1750, which is the alternative populate option the shared board already provides. Because each sensor sits behind a driver abstraction (subsection 4.6.5), this is a different population of the same design rather than a separate build.

The change of front end shifts what the room reports. The HMMD module detects presence and movement only; its 24 GHz wavelength is too long to recover heart rate or respiratory rate, which remain exclusive to the mirror-paired rooms and their 60 GHz MR60BHA2. The BMP280 reports temperature and pressure but not humidity, and the generic module carries no air-quality sensor. The GL5516, used for ambient-light reading to register when a room light is switched on or off, which feeds the tracking of the patient's activities of daily living. A generic room therefore contributes presence, temperature, and room-light activity, while heart rate, respiratory rate, humidity, and air quality are available only where a mirror-paired module is installed.

The variant exists so that the architecture scales beyond the single demonstrator room. Each additional generic module is provisioned with its own identity and certificate and publishes under its own `rmms/<UUID>/` namespace to the same tablet broker; the gateway forwards its readings as FHIR in the same way as for the mirror-paired module. No two-way mirror is involved, and the broker, the gateway, and the protocol are unchanged as modules are added.

4.1.3. Tablet

The tablet is an ITMEIPC IT-701A running a Termux environment, and it fills two roles at once. It hosts the Mosquitto broker through which all sensor traffic passes, and it runs the MagicMirror² display behind the two-way acrylic, where a bridge process subscribes to the broker and feeds a MagicMirror module that renders the vitals. The mirror subscribes only to its paired room's topics, so the generic modules reach the broker and the gateway without ever appearing on the display. The same device advertises itself on the local network by multicast DNS, so a module finds the broker by name rather than by a fixed address, and it couples radar presence to the screen so the mirror wakes only when someone is in front of it. The display software is described in section 4.2, the mirror materials in section 4.5, and the start-up and self-healing in Section 4.4.

4.1.4. Gateway

The aggregation and translation tier runs on a single-board computer. The demonstrator uses a Rock-Pro64, while the intended deployment board is the Radxa Dragon Q6A, whose neural accelerator is reserved for the future face-recognition workload of Section 2.5 or other edge computing tasks. A standalone service subscribes to the broker across every device namespace, decodes each sensor message, translates it into an HL7 FHIR Release 4 Observation, buffers it, and posts it to a FHIR server. This service never shares code with the firmware, and no clinical terminology runs on the module. The translation and delivery are detailed in section 4.3 and the service start-up in section 4.4.

4.1.5. Data Path

In normal operation each sensor task samples at its own rate: the environmental and air-quality sensors at roughly one hertz, the radar continuously and decimated to about one hertz, and the light sensor every few seconds. The transport task persists every reading to a small non-volatile flash buffer and then publishes it as JSON, inside the mutually authenticated TLS session, to a per-device topic on the tablet broker. The broker fans each message out to its subscribers, the mirror bridge that renders the tile and the gateway that builds and forwards the FHIR resource. Generic modules follow the same path, minus the mirror bridge. Two messages travel the other way: on each connection the tablet publishes the current wall-clock time to the module, which has no real-time clock of its own, and the operator station publishes the short text and page-selection messages that the mirror displays. The flash buffer at the module and the store-and-forward queue on the gateway mean that a dropped link or a brief power loss delays delivery rather than losing data.

4.1.6. Linux Environment Deployment Options

The MagicMirror platform and Mosquitto MQTT broker run in a Linux environment. There are several ways in which this can be achieved on an Android device:

1. Using an application called Termux, an open-source terminal emulator through which Linux commands can be run [42]. It presents little risk and requires little installation effort during deployment, can work across different types of tablets, and can be configured to start applications at boot using Termux:Boot. However, Android could shut down the applications or background processes due to i.e. battery optimization, which needs to be prevented in order to have a reliable remote health monitoring system.
2. By rooting the Android device, which provides administrator access and more control over background processes. However, the rooting process differs per device, and errors during the rooting process may render the device unable to boot or function correctly.
3. By installing a Linux operating system, which offers native support for Linux servers and improved system control. However, the installation process requires more effort and hardware support is limited to a select number of devices.

Due to the requirement of working across multiple devices and limited development time and materials during the project, the first option was implemented.

4.2. User Interface

4.2.1. Graphic Design of the User Interface

The smart mirror requires a simple and user-friendly display tailored to elderly users, which presents vital signs and environmental data. See section 2.1.1 for an overview of considerations when designing for older adults.

The user interface could present all sorts of information, aided by the modular design for the MagicMirror² platform. A smart mirror might for example contain a section which displays sensor measurements, a section with weather data, a section with a calendar, etcetera, thus adding functionality. However, because the screen of the available tablet is quite small (7-inches), the choice was made to create a single module which takes up the majority of the screen, to provide the sensor measurements to the user.

The specific sensors placed on the sensor module within the smart mirror measure heart rate, respiratory rate, movement, presence, (room) temperature, humidity, TVOC (Total Volatile Organic Compounds), AQI (Air Quality Index) and CO₂ concentration. All these values could technically be displayed on the screen. However, to keep the display simple and easy to understand, only the following are displayed: heart rate, respiratory rate, (room) temperature, humidity and air quality (according to the Air Quality Index), as these are relatively easy to interpret and to prevent an overload of information.

As the display is placed behind a two-way mirror, it uses a dark background to better show the reflection of the person standing in front of the mirror, with white text for high contrast.

Figure 4.2 shows an initial layout of the user interface. It contains information of the time and date at the top, followed by the vital signs, environmental measurements and a place holder for messages to the user. These messages could for example take the shape of instructions (*"please stand still for 5 seconds whilst measuring..."*), interpretation of the data (*"the air seems a bit stuffy, you could open a window for some fresh air?"*) or messages from the health care provider (*"don't forget to take your medicine at 5 o'clock"*).

The different sensors measurements are clearly indicated with an icon, name (e.g. heart rate, temperature etc), and a label to interpret the data for the user ("good", "moderate" and "bad"), thus making the interface self-explanatory; a first-time user would instantly know what information was presented, which is beneficial in general and specifically for users who experience memory loss.

However, there are two complications with this initial design. The first concern is the possibility of too much visual information, as elderly users can experience difficulty searching through visual information. The second concern is that the text is expected to be too small to be legible at arms-length on a 7" screen by elderly users.

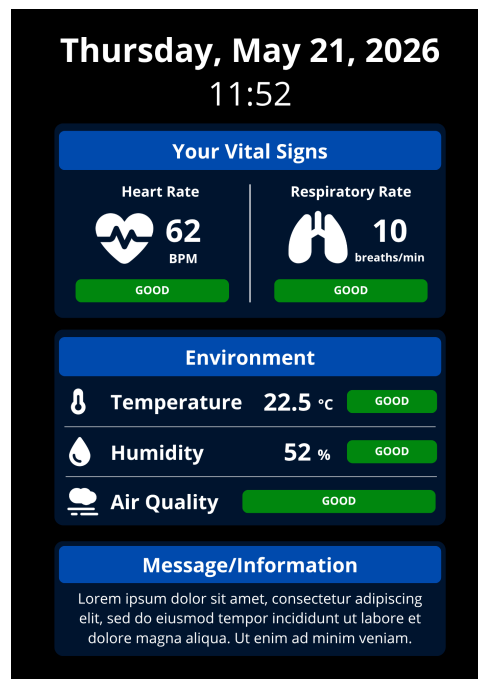


Figure 4.2: Initial design of the user interface.

Several design iterations resulted in the final design, see figure 4.3a. Most text has been removed, to reduce the visual information and increase the size of the remaining text and icons. Each sensor measurement is indicated by an icon, meant to be self-explanatory, a unit, and a symbol to interpret the measurement for the user with a traffic light system (green, yellow or red). Figure 4.3b shows an initial waiting screen before any measurements are received.

To show the possibility of adding additional screens and expanding the functionality of the smart mirror beyond its use within a remote health monitoring system, an additional screen was created as a place holder for video conferencing (see figure 4.3c).

All text on the screen uses the Tiresias font, specifically designed for legibility in case of impaired vision, as is common among older adults.

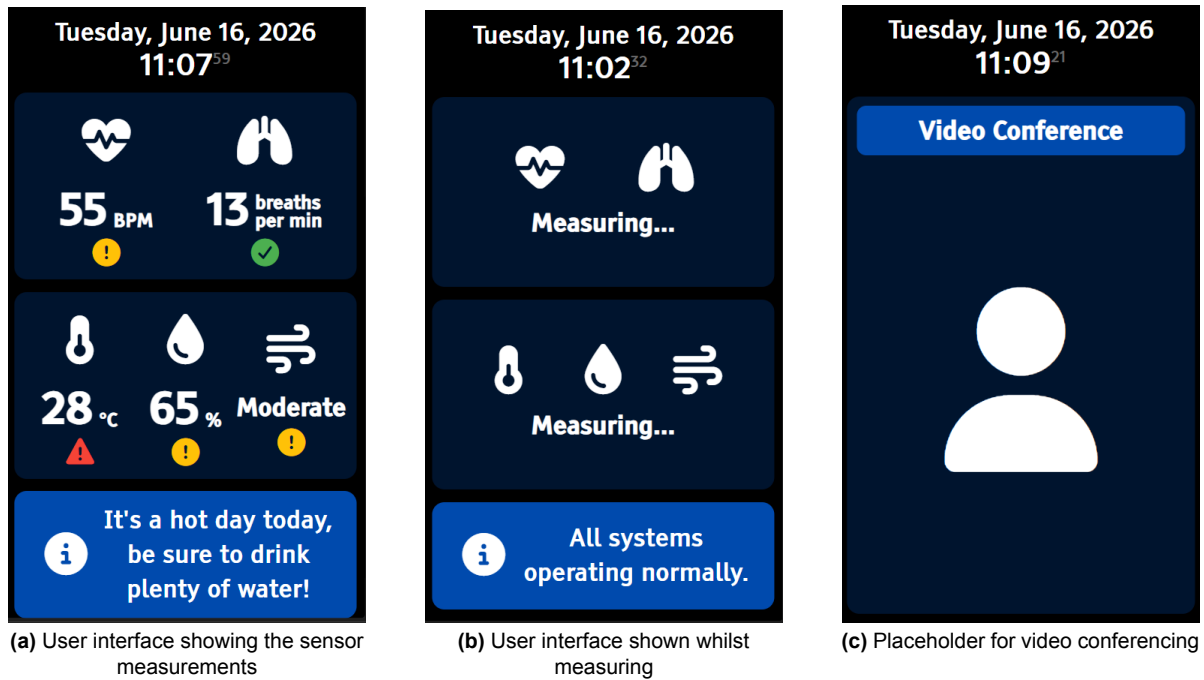


Figure 4.3: The final user interface design. The date and time are shown at the top, followed by a container to display heart rate and respiratory rate. The next container shows room temperature, humidity and air quality, with a placeholder for messages to the user in the bottom container.

4.2.2. Programming the User Interface

The UI was programmed using the open-source MagicMirror² platform (see [10] for more information). The platform makes use of different modules which can be placed across the screen, with a set of built-in modules (e.g. clock, weather and calendar) as well as the option to install third-party modules or create new modules conforming to the API. Communication between the modules takes place via notifications.

For the purposes of this project, three new modules were created: a *SensorUI* module to display the measurements on the screen, a *CustomMQTTBridge* which receives messages from the MQTT broker and sends them through a notification to *SensorUI*, and a *VidConf* module which displays a placeholder for video-conferencing. The *SensorUI* currently marks each sensor value with a green, yellow or red icon depending on its range. For example: a temperature of 20-24°C has a green icon, 18-20°C and 24-27°C yellow and lower than 18°C or greater than 27°C a red icon. These thresholds are currently hard-coded and chosen arbitrarily, and could in future be determined per client by a health care provider and set remotely via MQTT messaging.

The project also makes use of the default *clock* module to show the date and time, and the third-party module called *pages*, which provides the possibility to configure multiple pages with different module combinations and select a page based on a notification. This could be used in future to extend the functionality of the smart mirror beyond its use as a remote health monitoring device.

All custom modules make use of relative sizes for the visual components, to ensure the modules scale with the size of the tablet or device used. Additionally, the font Tiresias has been added to the project and used for every module.

4.2.3. Fullscreen User Interface Configuration

The MagicMirror server can be accessed through a web browser, by going to a specific address. To ensure a fullscreen view of the designed interface a kiosk browser was used, specifically Fully Kiosk Browser [43]. It can be launched automatically from Termux after starting the MagicMirror server.

4.3. Communication

Section 2.3 argued the protocol and security design and Section 4.1 gave the system overview. This section documents the communication as built, organised by the path a measurement takes: the topic and message contract that every party shares (Section 4.3.1), the secured channel and the access control that authenticate and confine each party (Section 4.3.2), and then the node's behaviour across outages, the command and time-synchronisation downlink, the gateway's translation of sensor JSON into HL7 FHIR, and the tests that verify the security claims, which the remaining subsections cover.

4.3.1. Topic Namespace and Message Schema

Every message in the system is an MQTT publication on a topic rooted at `rmms/<uuid>/<channel>`, where `<uuid>` is the device identifier and `<channel>` names the data stream. Topics carry no personally identifying information, no whitespace, and no diacritics, so the topic tree itself is safe to log. The contract is small and closed: a sensor node publishes four measurement channels, publishes its liveness on a fifth, and subscribes to exactly one command channel and one time channel. Table 4.2 lists the channels the node uses, together with two channels that live in the per-device tree but are produced and consumed entirely off the node, by an operator and by the mirror, and are included only so the tree is described in one place.

Table 4.2: MQTT channels in the per-device topic tree. The node publishes the uplink channels and subscribes to the two node-handled downlink channels; the operator-to-mirror channels are listed for completeness and are never touched by the node.

Channel	Direction (node)	Payload	QoS	Retained
env	publish	environmental sample (JSON)	1	no
air	publish	air-quality sample (JSON)	1	no
radar	publish	radar sample (JSON)	1	no
light	publish	light sample (JSON)	1	no
status	publish	"online" / "offline"	1	yes
cmd	subscribe	command (JSON)	1	no
time/set	subscribe	{"epoch_ms":...}	1	yes
log	publish	text line	0	no
info	operator → mirror	{"text":...}	not node-handled	
screen	operator → mirror	{"page":1--4}	not node-handled	

The per-channel quality of service follows the value of the data. The four measurement channels and the command channel use QoS 1, at-least-once delivery with a broker acknowledgement, because a lost reading or a lost command matters. The status channel is published retained at QoS 1 so that a subscriber learns the node's last known liveness immediately on connecting; it carries "online" on connect and, through the MQTT last-will-and-testament, "offline" if the connection drops. The log channel is best-effort at QoS 0.

Every measurement shares one JSON envelope, shown in Table 4.3, with a sensor-specific body in the `v` field. The envelope is what lets the downstream FHIR translation proceed with no clinical knowledge on the node: it carries a monotonic device timestamp, a wall-clock timestamp or a sentinel until the node is time-synced (Section 4.3.4), a per-channel sequence number that is persisted across reboots and is the basis for downstream deduplication (Section 4.3.5), and a quality flag the gateway maps to a FHIR status.

The sensor bodies are flat and fixed. The environmental body carries `temp_c` and `hum_pct`, with `pres_hpa` present but `null` because the chosen AHT21 has no pressure element; the air body carries `co2_ppm`, `tvoc_ppb`, and an integer air-quality index `aqi`; the radar body carries `presence` together with nullable `distance_mm`, `breath_bpm`, `heart_bpm`, and a breath-hold indicator `resp_motion`; the light body carries `lux`. The format is RFC 8259 JSON, compact, capped at 256 bytes per sample, with `snake_case` field names, a nesting depth of at most two, and no optional fields: a value that is un-

Table 4.3: Common JSON envelope carried by every measurement sample.

Field	Type	Meaning
ts_us	uint64	monotonic microseconds since boot; always valid
wall_ms	int64	epoch milliseconds, or -1 until the node is time-synced
seq	uint32	per-channel counter, persisted across reboots
q	uint8	quality: 0 ok, 1 stale, 2 degraded, 3 invalid
v	object	sensor-specific body

available is encoded as `null` or a documented sentinel rather than omitted, so a receiver never has to handle a field that is sometimes absent. The node encodes outbound JSON with fixed `snprintf` templates and parses the two inbound channels with a small tokenizer, so there is one serialiser and one parser and no dynamic allocation on the data path.

4.3.2. Transport Security and Access Control

The node reaches the broker over a single secured path: lwIP's MQTT client runs over `altcp_tls`, lwIP's mbedTLS binding, over TCP over the CYW43 Wi-Fi interface. There is no plaintext path and no alternative transport, so every byte the node sends travels inside one mutually authenticated TLS session under one trust model. The cryptography is fixed to elliptic-curve primitives: an ECDSA P-256 key pair for identity, ECDHE for key agreement, and AES-GCM for the record layer, restricted to the suites ECDHE-ECDSA-AES128-GCM-SHA256 and ECDHE-ECDSA-AES256-GCM-SHA384, with RSA, CBC, and SHA-1 excluded. The deployed handshake is TLS 1.2, because the pico-sdk's bundled mbedTLS compiles the 1.2 sources but not the 1.3 implementation in the version used; the configuration is written so that a later SDK enables 1.3 without other change. Table 4.4 collects the parameters.

Table 4.4: Transport-security parameters on the local-area leg.

Parameter	Value
Stack	lwIP MQTT over <code>altcp_tls</code> (mbedTLS) over TCP
Protocol	TLS 1.2 deployed; configuration 1.3-ready
Authentication	mutual; ECDSA P-256 certificates, one project CA
Key agreement	ECDHE (P-256)
Record cipher	AES-GCM (128 or 256), SHA-256/384
Excluded	RSA, CBC, SHA-1
Listener	port 8883, anonymous disabled, client certificate required
Identity field	certificate common name = device UUID
Resumption	TLS session tickets; renegotiation disabled

Authentication is mutual and anchored on one certificate authority. The node holds three artefacts written to its flash at provisioning time, the device certificate, its private key, and the authority certificate, whose creation is the subject of section 4.4.1; here they are used, not made. During the handshake the node validates the broker's certificate against the authority certificate, and the broker validates the node's certificate against the same authority, so neither side trusts a party the project authority did not sign. The node's certificate carries the device UUID as its common name, and that single field drives both identification and authorisation, as figure 4.4 shows.

The broker is Eclipse Mosquitto, listening only on port 8883 with anonymous access disabled, client certificates required, and session persistence enabled; its configuration is generated at provisioning time (Section 4.4.1) rather than hand-maintained. Two settings turn the certificate into an access-control identity. The first, `use_identity_as_username`, makes the broker adopt the certificate common name as the connection's username. The second is the access-control list, expressed once as a pattern over that username rather than per device, so that a node may write only under its own subtree and

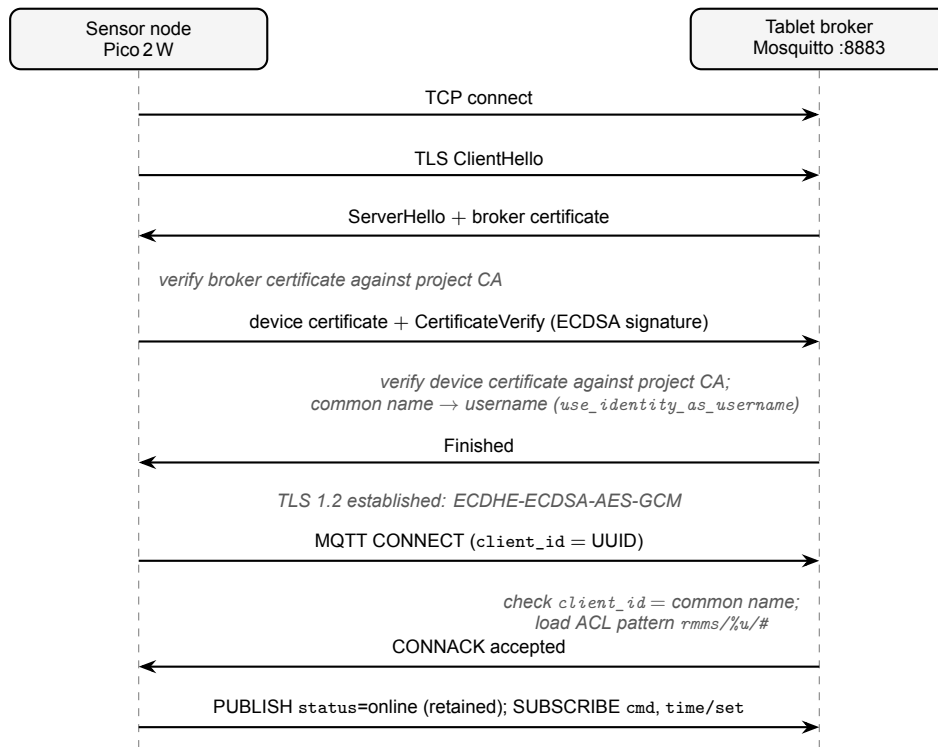


Figure 4.4: Connection sequence on the local-area leg: a mutual TLS 1.2 handshake in which each side validates the other against the one project certificate authority, after which the broker adopts the device certificate’s common name as the username, binds the MQTT client identifier to it, and applies the per-device access-control pattern. The device UUID is the single identity that flows through the whole exchange.

read only its own command channel,

```
pattern write rmms/%u/#    pattern read rmms/%u/cmd
```

where `%u` expands to the username, and therefore to the common name, and therefore to the device UUID. Keying the pattern on the username (`%u`) rather than on the MQTT client identifier (`%c`) is deliberate: the common name is cryptographically bound to the certificate the broker has just verified, whereas the client identifier is a self-asserted connection field, so `%u` ties authorisation to authenticated identity. The two non-node identities issued by the same authority are scoped the same way: the mirror certificate (common name `mirror-<id>`) is granted read access across the device tree so it can render every node’s tiles, and the operator certificate (common name `operator-<id>`) is granted write access only to the `info` and `screen` channels. The effect is per-device isolation with no per-device broker configuration, because the pattern already covers a device the moment its certificate is trusted.

On reconnect, mbedTLS resumes the session from a TLS session ticket where the broker offers one, which shortens a dropped-link reconnect from a full handshake to a resumption; renegotiation is disabled, so a session is torn down and re-established rather than renegotiated in place. The behaviour of the node around those reconnects, and the buffering that protects data across them, is the subject of Section 4.3.3.

4.3.3. Sensor-Node Client and Outage Handling

The node’s MQTT client is lwIP’s built-in version 3.1.1 client, driven by a single transport task that is the only consumer of the four producer queues into which the sensor tasks place their samples. Keeping one task on the network path means the TLS context, the publish path, and the reconnect logic share one thread and need no locking among themselves, and all lwIP calls are made under the TCP/IP core lock the port provides.

The node finds the broker by name rather than by address. The broker host is provisioned once as

`tablet.local`, and the node resolves it through lwIP's resolver; with multicast-DNS queries enabled, a `.local` name is resolved by a multicast query on the local segment rather than by a unicast DNS server, and the tablet answers it, the tablet running the responder and the node only querying. Because the name is stable, the provisioned configuration never has to change across networks or address changes. Where multicast DNS is unavailable on a given network, the documented fallback is to re-provision the configuration with a literal address; the previous design's unauthenticated UDP broadcast discovery is not used and is not reintroduced.

Delivery is made loss-tolerant by a persistent buffer rather than by hope. Every sample is first written to a non-volatile flash spool, a first-in-first-out log held in a dedicated flash region separate from the configuration store, before any attempt to send it. The transport task drains the spool and clears a record only when its QoS 1 publication is acknowledged by the broker, retrying across reconnections, so a network outage or a power cut loses no data up to the spool's capacity of at least fifteen minutes; if the spool fills, the oldest records are dropped and the event is logged rather than failing silently. The sequence number in each record is persisted across reboots, which is what lets the gateway deduplicate correctly even after the node restarts (Section 4.3.5). Figure 4.5 shows the path.

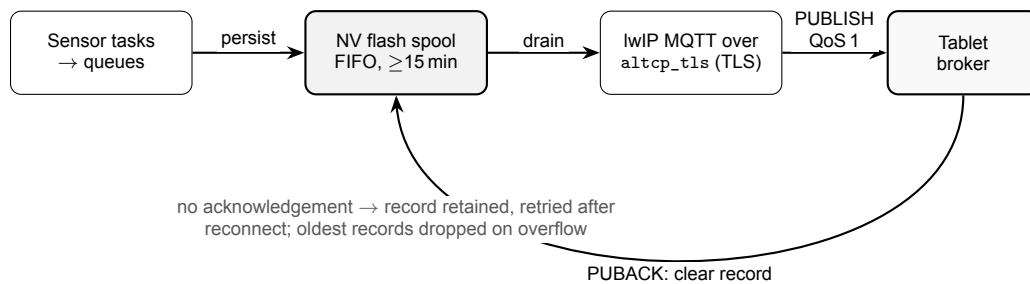


Figure 4.5: Uplink with the non-volatile flash spool. Every sample is persisted before transmission and cleared only on its QoS 1 acknowledgement, so an outage or power loss delays delivery rather than losing it.

Around this, the connection management is conservative. The keepalive interval is sixty seconds. On a dropped link the node reconnects with exponential backoff from one second to thirty-two seconds and then at a constant thirty-two seconds, so an extended outage does not turn into a reconnect storm against the broker. Liveness is published through the retained status channel and the last-will-and-testament described in Section 4.3.1: the broker holds "offline" as the will and the node replaces it with "online" on every successful connect, so any subscriber, the mirror included, sees an accurate last-known state immediately on subscribing.

4.3.4. Command and Time Synchronisation

The node subscribes to two downlink channels and treats both as closed contracts. The command channel carries a fixed set of four commands, `activate` and `deactivate` to start and stop publishing, `deregister` to clear the device's configuration while leaving its identity certificates intact, and `shutdown` to drop the links and idle until a power cycle; anything outside that set is ignored. Commands are delivered at QoS 1 and are not retained, so there is no retained command left on the broker to be replayed against a node that reconnects later.

The node has no real-time clock, so wall-clock time is supplied by the tablet. The node subscribes to the time channel on connecting, and the tablet publishes the current epoch time to that channel, retained per device, when it observes the device connect; the node maintains a wall-clock offset from this and stamps each spooled record's wall-clock field at the moment it is buffered. Until the first time message arrives the field carries a sentinel, and the gateway substitutes a receive-time estimate and marks such an Observation preliminary (Section 4.3.5). No time is taken from the wide-area network. Figure 4.6 shows both flows.

4.3.5. Gateway Aggregation and FHIR Translation

The translation from sensor data to clinical data happens entirely on the gateway, a standalone service that shares no code with the firmware and runs on the single-board computer. It is an MQTT subscriber

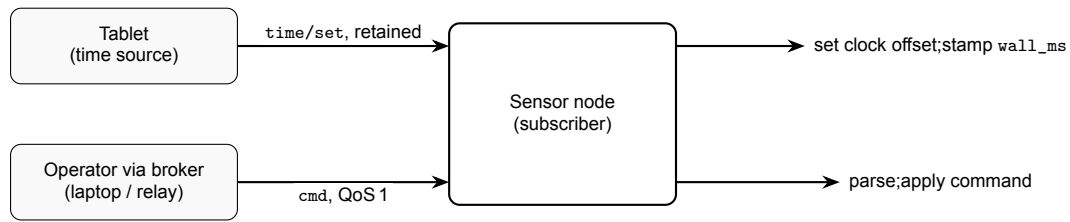


Figure 4.6: The two downlink flows. The tablet supplies wall-clock time on the retained time channel, which the node turns into a clock offset; an operator (the laptop in the proof of concept, a relay in production) sends commands from the closed set {activate, deactivate, deregister, shutdown} on the command channel.

authenticated to the broker by a certificate from the same project authority, so it is one more mutually authenticated client on the local-area leg rather than a privileged component. No FHIR resource, no LOINC code, and no clinical terminology exists on the node; the node emits only the compact sensor JSON of Section 4.3.1, and the gateway is where that becomes HL7 FHIR.

The pipeline, shown in Figure 4.7, is the same for every sample. The gateway decodes the JSON into a typed internal object, deduplicates it on the triple of device, channel, and sequence number, and builds a FHIR Release4 Observation from it. Heart rate and breath rate are emitted as LOINC-coded vital-signs Observations, using 8867-4 for heart rate and 9279-1 for respiratory rate with the corresponding UCUM units; the ambient channels, environmental, air-quality, and light, are forwarded as Observations but are not mapped to standard vital-signs codes, consistent with the absence of widely-adopted codes for indoor ambient measurements noted in Section 2.3.6. The envelope fields populate the resource: the device UUID becomes the *Device* reference, the gateway’s own binding of device to resident supplies the *Patient* reference and is the only place that binding exists, the wall-clock field becomes the effective time, or a receive-time estimate marking the Observation preliminary when the node was not yet time-synced, the quality flag maps to the Observation status, and the sequence number becomes a business identifier under a project-specific system.

Each built Observation is validated before it can advance. A valid Observation enters a local store-and-forward queue, held for at least twenty-four hours, and is posted to the medical server in a transaction Bundle; an invalid or unmappable payload is routed to a dead-letter store with its raw JSON retained, never silently dropped. Idempotency spans the whole chain: the sequence-number identifier lets the gateway post with a conditional update keyed on that identifier, so that a retry after a gateway restart or a medical server-side outage updates the existing resource rather than creating a duplicate, and the same constraint is enforced locally as a uniqueness key on the triple of device, channel, and sequence number. Because the node persists its sequence numbers across reboots (Section 4.3.3), this deduplication holds even when a node restarts mid-stream.

The wide-area leg itself is exercised at the level of the FHIR application interface against a reference HAPI FHIR server: the gateway constructs valid resources and posts them over HTTPS. Authentication of that leg is out of the current scope, as Section 2.3.7 states, and the reference server accepts the posts without it.

4.3.6. Verification

Two independent checks establish that the security design is enforced and not merely configured. The first is a scripted access-control harness run against the live tablet broker, which exercises the access-control list from the perspective of each certificate identity and asserts both the permitted and the denied cases. Its seven assertions, listed in Table 4.5, all pass: a device can write its own subtree and read its own command channel but is denied any other device’s topics, an operator can write only the display channels and is denied a sensor channel, and a connection presenting no client certificate is refused outright. Because each negative case is the clean proof, this confirms that the per-device isolation of Section 4.3.2 is actually imposed by the broker rather than assumed.

The second check is the firmware itself. The sensor-node firmware completes the mutual-TLS handshake against the same broker and publishes its measurement channels through the enforced access-control list, and the broker’s log records the negotiated session as TLS 1.2 with an ECDHE-ECDSA

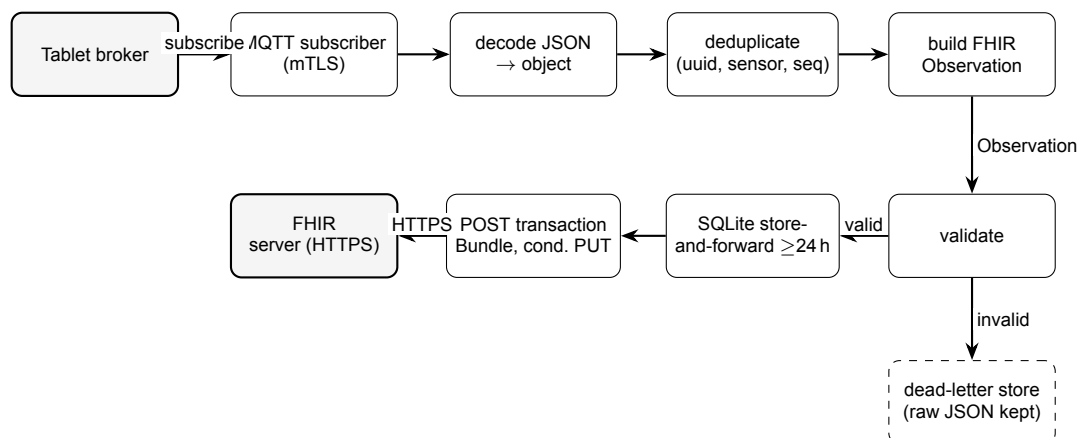


Figure 4.7: Gateway translation pipeline. Sensor JSON is decoded, deduplicated, and built into a FHIR R4 Observation (heart rate and breath rate LOINC-coded, ambient channels forwarded uncoded), validated, queued in a 24-hour store-and-forward buffer, and posted to the medical server in a transaction Bundle with conditional-update idempotency. Invalid payloads are dead-lettered rather than dropped.

Table 4.5: Access-control assertions checked against the live broker, all passing.

#	Action	Expected	Result
1	Device publishes to its own subtree	accepted	pass
2	Device publishes to another device's subtree	denied	pass
3	Device subscribes to its own command channel	accepted	pass
4	Device subscribes to another device's command channel	denied	pass
5	Operator publishes to a display channel	accepted	pass
6	Operator publishes to a sensor channel	denied	pass
7	Connection without a client certificate	refused	pass

suite on the P-256 curve and the device certificate's common name adopted as the username. The access-control harness uses reference command-line clients and the firmware exercises the same path from the embedded stack, so together they cover both broker-side enforcement and firmware interoperability over one secured channel.

4.4. Automation and Deployment

4.4.1. Tablet Deployment Script

The tablet, which hosts both the system's MQTT broker and the MagicMirror² interface, is provisioned by a single on-device script (`deploy.sh`) that installs the broker, the MagicMirror² interface and the supporting services, and registers them to start automatically whenever the tablet boots. A second, short step run from the workstation then delivers the tablet's mutual-TLS certificates, which for security reasons are never generated on the device itself. The full procedure and its design rationale are described in Appendix D.

4.4.2. Display Activation and Deactivation

The display of the smart mirror need only be powered when a person is in front of it. Most of the time, the screen can and should be deactivated, as prolonged activation of the display drains power and causes burn-in in case of a stationary image. Additionally, a client might object to a wall-mounted display which is powered on at all times of day.

To determine when the screen should be (de)activated, the sensor module sends a "display on/off" signal to the MQTT broker, based on the presence measurements of the radar sensors.

As the screen of the tablet cannot be turned on or off via Linux commands in Termux on a non-rooted device, a supplementary Android application was designed. This application can lock the device on command, thus turning off the screen, and wake the display on command. The application has several components: *BootReceiver*, *DeviceAdminReceiver*, *WakeService*, *MQTTService* and *MainActivity*.

- The *BootReceiver* receives a signal when Android finishes booting, and starts the *MQTTService*.
- The *MQTTService* connects to the MQTT broker at a local address (as the broker runs on the same device), subscribes to the topic "*display*" and listens to any incoming messages. Receiving an "ON" starts the *WakeService*. Receiving an "OFF" executes a command to lock the device, which automatically turns off the screen. *MQTTService* also creates a notification and runs as a foreground service, to keep the service running and prevent it from being shut down by Android (due to for example battery optimization).
- The *WakeService* is a foreground service to wake the screen after a device is locked. Note that the *WakeService* does not unlock the device, as third-party applications are not authorized to do so as this presents a security risk. This is circumvented by configuring the device without setting a password or locking mechanism.
- The *DeviceAdminReceiver* ensures the application has Device Administrator privileges, which is necessary to lock the device.
- The *MainActivity* handles the UI of the application, and is activated when a user manually starts the application. This interface is not necessary for the remote health monitoring system, but is useful during development and debugging. The *MainActivity* displays messages broadcast by the *MQTTService*, creates an "ON" button which starts the *WakeService*, and creates an "OFF" button which executes the command to lock the device. See figure 4.9 for a view of the application.

A diagram of the automated operation of the application is shown in figure 4.8.

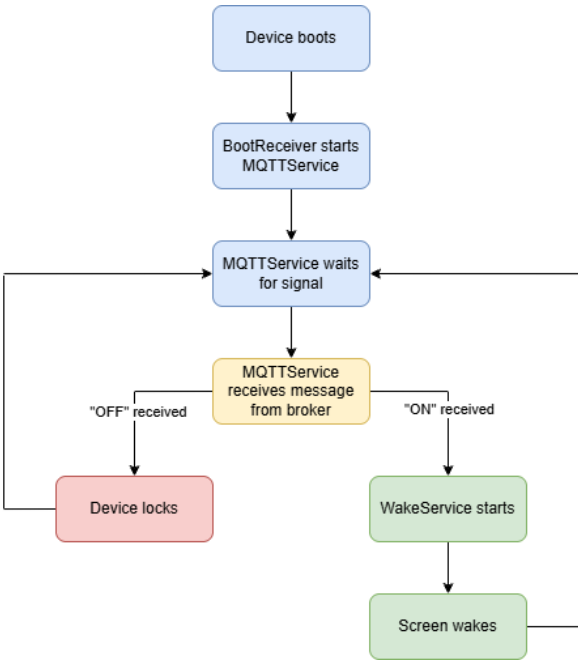


Figure 4.8: Diagram of the functionality of the Android application responsible for turning the screen on/off after receiving a message from the MQTT broker.



Figure 4.9: The visual interface of the Android application responsible for turning the screen on/off after receiving a message from the MQTT broker.

4.5. Mirror Materials

A smart mirror consists of an electronic display behind a two-way mirror. If the display is off, the object looks like a regular mirror. If the display is on, its light passes through the mirror and the user can see his or her reflection as well as the information on the display, thus providing an interface for elderly users in an object that fits into a home and naturally encourages a person to stand in front of it, which aids in taking the necessary vital sign measurements.

The mirror consists of an acrylic sheet onto which a two-way mirror film is applied. Acrylic was selected instead of glass because it can be precisely laser-cut locally, allowing openings to be created directly in front of the radar sensor within a sensor module. These openings reduce the interference caused by the mirror assembly, thereby improving the accuracy and reliability of the radar measurements. The acrylic plate has a thickness of 3mm, as this material was readily available.

A tablet is mounted behind the mirror to provide the display functionality of the smart mirror. The areas of the mirror not covered by the tablet are backed with black paper to enhance the perceived reflectivity of the mirror and improve its visual appearance.

4.6. PCB Design and Component Integration

The objective of the PCB design was to integrate the required sensing, display, power, and control electronics into one compact and modular hardware platform. The PCB was designed to support environmental sensing, radar sensors measurements, light-intensity measurement and visual feedback. The main controller of the system is a Raspberry Pi Pico 2WH module, which interfaces with environmental sensing modules, radar sensing modules, a light-dependent resistor, an status OLED display, two push buttons, and two status LEDs.

The PCB includes support for environmental sensors such as the ENS160 + AHT21, with the possibility of using a BMP280 sensor module depending on the final configuration. These sensors provide the following environmental data:

- ENS160: equivalent CO2 values, Air Quality Index (AQI) and Indoor Air Quality (IAQ) [44]
- AHT21: Temperature and humidity [45]
- BMP280: Temperature and pressure [46]

In addition, the PCB supports radar modules for either vital-sign measurement (MR60BHA2/C1001) or distance/presence detection (HMMD). Although Section 2.2.4 establishes that the MR60BHA2 is the superior alternative, the C1001 radar module was deliberately integrated into the PCB design for two primary reasons. First, a previous research group encountered unresolved 'ghost readings' with the C1001; incorporating it into this prototype provides an opportunity to investigate and diagnose the root causes of these anomalies. Second, because the C1001 shares the same spatial footprint as the MR60BHA2, its inclusion did not require any compromises regarding PCB real estate or layout constraints."

A light-dependent resistor is included to measure ambient light intensity, and an status OLED display is used to display system information and measurement status.

KiCad was selected as the PCB design environment because it is open-source, free to use, versatile, and widely supported by a large electronics-design community. It provides schematic editor, PCB layout, 3D visualisation, and support for both standard and custom footprint libraries [47]. This was important for this project because several modules required custom or modular footprints. In the schematic, net labels were used instead of long traditional wires where possible A.1. This improves readability, reduces visual clutter, and makes it easier to follow functional connections such as UART, I2C, power rails, and test points.

4.6.1. Modular Radar Footprint and Antenna Keep-Out Strategy

A major design requirement was that the PCB should support the components from the program of requirements in a modular way. This was especially important for the radar section, where the following radar modules had to be considered: C1001/MR60BHA2, and HMMD . Because these modules do not have identical mechanical outlines and antenna positions, their placement required careful consideration. To make the PCB modular, overlapping component outlines were used in the radar area. Although this creates a visually busier footprint, it allows different radar modules to be evaluated without redesigning the entire PCB.

The antenna area of the radar modules was one of the most critical layout constraints. Radar antennas are sensitive to nearby copper, ground planes, tracks, vias, and other conductive objects. Copper underneath or close to the antenna can disturb the antenna radiation pattern, reduce sensitivity, change the effective impedance, or create reflections. For this reason, antenna areas were kept free from copper as much as possible.

The MR60BHA2 radar module received the highest priority in the keep-out design because it was selected for vital-sign measurement. This module is intended to detect small movements related to respiration and heartbeat, which makes it more sensitive to antenna disturbance than a simple distance-measurement application. Therefore, a dedicated antenna keep-out zone was added to the modular MR60BHA2 footprint. This keep-out zone prevents copper pours, tracks, vias, and pads from being placed in the antenna area. By enforcing this constraint directly in the PCB footprint, the design reduces the risk that later routing changes accidentally introduce copper into the antenna region.

The trade-off of this approach is that the modular radar footprint becomes larger and contains overlapping outlines 4.10a. However, this trade-off is justified because it increases the flexibility of the prototype. It allows multiple radar options to be tested while still protecting the antenna area of the most critical radar module.

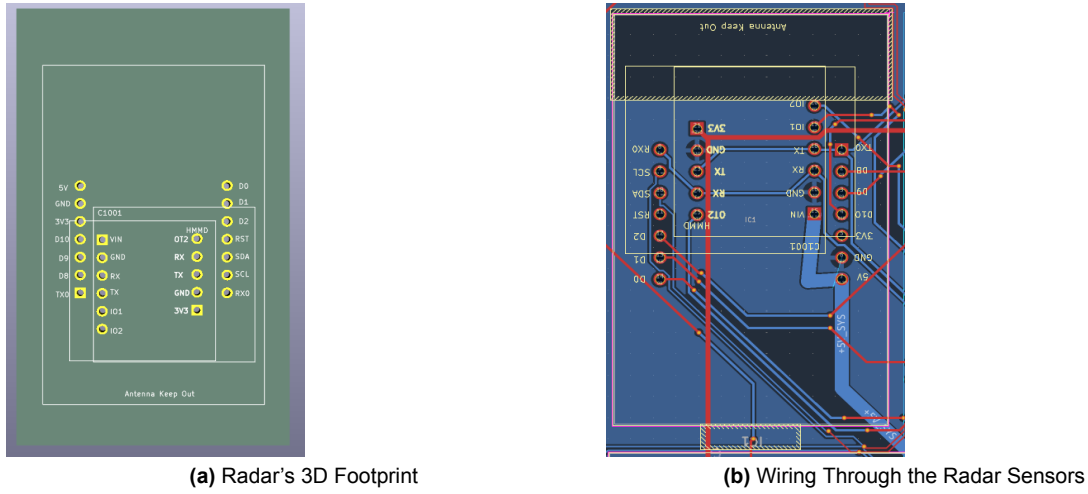


Figure 4.10: Radar Sensors Footprint Design

4.6.2. Power Architecture

The PCB was designed to accept either a 12 V input or an external 5 V input. This makes the board more flexible during testing and integration. When 12 V is used, the voltage is converted to 5 V using a buck converter based on the Texas Instruments LM2596T-5. When an external regulated 5 V supply is available, this can be connected directly to the 5 V input. A three-pin jumper is used to select whether the system 5 V rail is powered from the buck converter output or from the external 5 V input.

The 12 V input first passes through a polyfuse. The purpose of the polyfuse is to provide resettable overcurrent protection. If a short circuit or excessive current draw occurs, the resistance of the polyfuse increases, limiting the current and reducing the risk of damage to the PCB, wiring, or power supply. After the polyfuse, a Schottky diode is used for reverse-polarity protection. If the input supply is accidentally connected with the wrong polarity, the diode prevents reverse current from flowing into the circuit. A Schottky diode was chosen because it has a lower forward voltage drop than a standard silicon diode, which reduces unnecessary power loss and heat generation.

The LM2596T-5 [48] was selected instead of the previously considered L7805 [49] linear voltage regulator. The L7805 is simple and requires fewer external components, but as a linear regulator it dissipates the voltage difference between input and output as heat. For a 12 V input and 5 V output, the voltage drop across the regulator would be 7 V. The power dissipated by a linear regulator is approximately:

$$P_{loss} = (V_{in} - V_{out}) \cdot I$$

For example, at a load current of 500 mA, the L7805 would dissipate approximately:

$$P_{loss} = (12V - 5V) \cdot 0.5A = 3.5W$$

This amount of heat is undesirable in this design because the PCB includes environmental sensors. Temperature and air-quality measurements can be influenced by local self-heating from nearby power electronics.

In a Texas Instruments thermal comparison study [50], a 24 V to 5 V, 100 mA conversion test was performed. The switching buck converter dissipated only 0.093 W, while the linear regulator dissipated 2.06 W. The same study reported a lower temperature rise for the switching regulator: 11°C compared with 27°C for the integrated linear regulator. Therefore, the evidence supports the claim that a switching step-down regulator such as the LM2596 produces significantly less heat than a linear regulator such as the L7805.

4.6.3. Buck Converter Component Selection

The buck converter circuit was implemented using the LM2596T-5 fixed 5 V step-down regulator. The surrounding components were selected according to the typical application recommendations in the LM2596T-5 datasheet [48].

At the input of the buck converter, a bulk input capacitor is connected between the converter input and ground. This capacitor provides local energy storage and reduces input voltage ripple caused by the switching current of the converter. It also helps stabilise the input voltage when the load current changes. A smaller capacitor is placed in parallel with the bulk capacitor close to the regulator input. The large capacitor is effective at lower-frequency ripple and load transients, while the smaller capacitor improves high-frequency decoupling. This is important because switching regulators generate fast current pulses that must be locally bypassed to reduce noise and improve stability.

At the switching output of the LM2596T-5, a Schottky catch diode is connected to ground. This diode provides a current path for the inductor when the internal switch of the regulator turns off. Without this diode, the inductor current would have no controlled path, causing large voltage spikes and possible damage to the converter. A Schottky diode is preferred because of its fast switching behaviour and low forward voltage drop.

The inductor is one of the central components of the buck converter. It stores energy while the internal switch is on and releases energy to the output while the switch is off. Together with the output capacitor, the inductor smooths the pulsed switching waveform into a stable DC output voltage. In this design, a 33 (μ)H inductor was used, matching the recommended design region for a 5 V LM2596 application with a 12 V-class input supply.

An output capacitor is connected from the 5 V buck output to ground. This capacitor reduces output voltage ripple and improves the transient response when the load current changes. The output capacitor is also important for regulator-loop stability. The feedback pin of the fixed 5 V LM2596T-5 is connected to the output node, allowing the regulator to monitor the output voltage and maintain the regulated 5 V output.

The output of the buck converter is connected to one side of a three-pin jumper. The other selectable input of the jumper is the external 5 V input. The middle pin of the jumper connected to the system supply rail, named $+5V_{SYS}$. A power-indicator LED with a 1 k Ω resistor is connected to the selected $+5V_{SYS}$ rail. This means the LED indicates that the board is receiving the selected 5 V system supply, regardless of whether that supply comes from the buck converter or from the external 5 V input.

4.6.4. Microcontroller, Sensors, and User Interface

The Raspberry Pi Pico 2WH module acts as the central controller of the PCB. It communicates with the radar modules through UART using the Tx and Rx lines. The environmental sensor module and the status OLED display are connected through the I2C bus.

The status OLED display is used to show system information, measurement values, and status messages. A dedicated push button is included to control the information shown on the display. In the PCB design, this display-control button is connected as an active-low digital input. A 10 k(Ω) resistor pulls the input up to 3.3 V when the button is not pressed. When the button is pressed, the input is connected to ground. The microcontroller can detect this change and use it to switch between display pages or operating modes. A small series resistor is included between the button node and the microcontroller pin to provide additional protection in case the pin is accidentally configured as an output.

A separate reset button is connected to the RUN pin of the Raspberry Pi Pico module. Pressing this button pulls the RUN pin to ground, causing a hardware reset of the microcontroller. This is useful during development and testing because it allows the firmware to be restarted without disconnecting the power supply.

The PCB contains status LEDs for user feedback. Both LED's are connected to the microcontroller GPIOs through a 330 (Ω) resistors. One LED can be used to indicate a successful WiFi connection. Another is used as an additional status indicator.

4.6.5. PCB Layout Considerations

The PCB was designed as a two-layer board. A copper zone on the bottom layer was used as the main ground plane. This improves the ground connection between components, reduces the impedance of the return path, and simplifies routing. Thermal relief pads were selected for pads connected to the ground plane. Thermal reliefs make soldering easier because the pad is not directly connected to a large copper area on all sides. Without thermal reliefs, the copper plane can draw heat away from the pad during soldering, making reliable solder joints more difficult to achieve.

The placement of components was an important part of the PCB design. The radar modules, environmental sensors, display, LDR, buttons, and visible LEDs were placed on the front side because these components either interact with the environment or need to be visible and accessible to the user. The Raspberry Pi Pico module and the buck converter circuit were placed on the bottom side. This reduces the required area and keeps the user-facing side focused on sensing and interaction.

Placing the buck converter away from the environmental sensors also helps reduce the risk that heat from the power circuit affects the sensor readings. Although the LM2596T-5 is more efficient than a linear regulator, it can still generate heat depending on load current and input voltage. Therefore, the decision was made to place the buck converter as far as possible from the environmental sensing modules.

The final PCB dimensions are approximately 86.3 mm by 89.8 mm.

4.6.6. Test Points

Several test points were added to the PCB to simplify debugging, validation, and firmware development. Test points were included for Rx, Tx, SCL, SDA, V_{IN_BUCK} , and V_{OUT_BUCK} . Adding these test points improves the maintainability and testability of the PCB. They make it possible to isolate problems during bring-up, such as incorrect supply voltage, communication errors, or soldering faults

4.7. GL5516 Calibration Method and Result

To preserve consistency between the two light sensors used in the system, the GL5516 was calibrated against the BH1750. The BH1750 was used as a practical reference sensor to align the GL5516 readings within the demonstrator. Since the GL5516 is an analogue LDR, its resistance does not change linearly with light intensity. Therefore, a calibration curve was required to convert the measured LDR resistance into an estimated lux value.

A log-log calibration model was used because LDRs commonly show a non-linear power-law relationship between resistance and light intensity. This approach is supported by Marinho et al., who demonstrated that calibrated LDRs can be used as viable light-measuring sensors for illuminance measurements [51]. By fitting the constants (a) and (b), the GL5516 output was adjusted to produce values comparable to the BH1750, allowing both sensors to be used consistently within the larger system.

Table 4.6: Calibration measurements for the GL5516 using the BH1750 as reference.

Condition	BH1750 [lx]	ADC value	V_{ADC} [V]	R_{LDR} (Ω)
Room light	1030.8	3552	2.838	1626.9
Strong light	7747.5	3938	3.173	398.7

The calibration model was defined as:

$$\log_{10}(L) = a \cdot \log_{10}(R_{LDR}) + b$$

Using the two valid calibration points, the fitted parameters were:

$$a = -1.43436$$

$$b = 7.61944$$

Therefore, the final conversion equation used for the GL5516 is:

$$L_{\text{estimated}} = 10^{7.61944} \cdot R_{\text{LDR}}^{-1.43436}$$

or:

$$L_{\text{estimated}} \approx 41632775.87 \cdot R_{\text{LDR}}^{-1.43436}$$

This equation converts the calculated LDR resistance into an estimated light intensity in lux. Since the resistance of the LDR decreases when the light intensity increases, the calibration coefficient a is negative.

4.8. HMMD Filtering and Usage

The Waveshare HMMD radar sensor was selected as the presence detection sensor for this project. Its role is to support a wake-up signal for the microcontroller unit when a person is detected. This wake-up signal will allow the rest of the system, including the environmental sensors, tablet, and OLED display, to start only when needed.

The HMMD sensor provides both a digital presence output and UART-based measurement data. The OT2 pin is used as a fast presence indicator: it goes high when presence is detected and low when no presence is detected [20]. However, OT2 alone does not provide distance information. Therefore, the final wake-up decision is based on a combination of OT2, the UART presence output, and the measured target distance.

In report mode, the radar divides the space in front of it into distance gates. Each gate contains an energy value that represents the strength of the reflected radar signal in that range. One gate corresponds to approximately 70 cm; for example, gate 0 covers roughly 0–70 cm, gate 1 covers 70–140 cm, and gate 15 extends to approximately 1120 cm [20]. The measured distance can fluctuate due to factors such as body size, posture, radar cross-section, and reflections [20]. For this reason, raw measurements are not used directly.

During testing, gate 0 was found to be noisy and not useful for the intended detection range. Since the required activation zone starts at approximately 90 cm, gate 0 is ignored in the filtering logic. Gates 1 and 2 are used only as supporting evidence, while the UART target distance is used for the actual distance decision.

The filtering method was implemented as a sequence of validation and smoothing steps. First, gate 0 was excluded, and the strongest energy gate was determined only from gates 1–15:

$$g_k = \arg \max_{i \in \{1, \dots, 15\}} E_{k,i}$$

where $E_{k,i}$ is the energy value of gate i at sample k . A distance sample d_k was accepted only when OT2 was high, UART presence was reported, the strongest useful gate was either gate 1 or gate 2, and the raw distance was within the valid range.

To reduce noise, the accepted distance was smoothed using an exponential moving average. The EMA was initialized only after two consecutive stable raw readings were observed. After initialization, sudden jumps were rejected when:

$$|d_k - d_{k-1}| > 50 \text{ cm}$$

If three consecutive jumps were rejected, the EMA was re-initialized at the new distance to prevent the filter from locking onto an outdated value. The EMA was implemented using $\alpha = 0.25$:

$$\hat{d}_k = 0.25d_k + 0.75\hat{d}_{k-1}$$

This gives more weight to recent measurements while still reducing short-term fluctuations. The wake-up condition was accepted only when the smoothed distance remained within the activation range for three consecutive frames:

$$90 \leq \hat{d}_k \leq 150 \text{ cm}$$

To avoid flickering near the threshold, hysteresis was applied. After activation, the signal remains active as long as the smoothed distance stays within the wider range of 80–170 cm. The output is switched off only after this condition fails for six consecutive frames.

The current detection logic accepts a person only when the following conditions are satisfied: OT2 is high, the UART output reports presence, the target distance is within the configured range, and the relevant energy is mainly observed in gates 1 or 2.

An exponential moving average is applied to smooth the measured distance, followed by hysteresis and consecutive-frame confirmation to reduce flickering.

The resulting filtered decision is then used to generate a clean GPIO wake-up signal for the microcontroller. In the current configuration, the system is activated only when a person is detected between approximately 90 cm and 150 cm from the radar.

4.9. Filtering Method for Improving Reliability of MR60BHA2 Vital-Sign Output

The MR60BHA2 radar module internally processes the radar reflections and transmits decoded values, such as presence, distance, breathing rate, and heart rate, to the microcontroller over UART. Therefore, the implemented filtering algorithm does not estimate vital signs directly from raw radar phase data. Instead, it improves the reliability of the already processed radar output by rejecting unstable measurements before smoothing and reporting them.

The filtering architecture consists of seven stages, with frame validation and raw-value storage treated as the first combined stage: frame validation, raw-value storage, presence confirmation, distance stability validation, vital-sign plausibility validation, exponential smoothing, timeout reset, and final robust estimation.

1. Frame validation and raw-value storage

The radar sends binary UART frames. Each frame is first checked using the radar protocol checksum before its payload is accepted. Only valid frames are decoded and stored as raw values:

$$x_{\text{raw}} \in \{d, HR, BR, P\}$$

where d is distance, HR is heart rate, BR is breathing rate, and P is the human-presence flag. The raw values are not printed immediately. They are first passed through the reliability filter. This separation is important because communication validity does not automatically mean physiological validity.

2. Presence confirmation

A single human-detection frame is not considered sufficient evidence of a real target. Short movements and reflections can cause false detections. Therefore, the algorithm uses a temporal debounce mechanism. Presence is accepted only if evidence remains available for a fixed confirmation time:

$$t_{\text{now}} - t_{\text{candidate}} \geq T_{\text{presence}}$$

where T_{presence} is set to 5 s in the implementation. Presence evidence is defined as either a recent human-detection frame or a recent valid distance measurement inside the allowed range. This reduces short false positives because a brief detection does not survive the confirmation interval.

Absence is also delayed using an absence confirmation time, which was set to 4 s. This prevents the output from rapidly switching between “target present” and “no target” when the radar temporarily misses frames.

3. Distance gate and distance stability

Vital signs are only meaningful when the radar is observing a stable target at a suitable distance. The distance sample d_k is first checked against the allowed measurement range:

$$d_{\min} \leq d_k \leq d_{\max}$$

In the implementation, this range is set to:

$$40 \text{ cm} \leq d_k \leq 150 \text{ cm}$$

A jump rejection rule is then applied:

$$|d_k - d_{\text{cand}}| \leq \Delta d_{\max}$$

where d_{cand} is the current distance candidate and $\Delta d_{\max} = 20 \text{ cm}$. If the distance changes more than this threshold, the algorithm assumes that the radar target is unstable or that the radar has locked onto a different reflection. The distance confirmation timer is then restarted.

Only after the distance remains stable for $T_{\text{distance}} = 5$ s are vital-sign values allowed to pass through the next filter stage.

4. Vital-sign plausibility and jump rejection

Heart rate and breathing rate are filtered using prototype plausibility limits. The accepted heart-rate range is:

$$45 \leq HR_k \leq 125 \text{ bpm}$$

The accepted breathing-rate range is:

$$0 \leq BR_k \leq 30 \text{ breaths/min}$$

The reason why 0 is accepted is because a sanity check might be conducted where the subject holds their breath to verify whether the radar sensor detects the absence of breathing. Sudden changes are also rejected:

$$|HR_k - HR_{\text{cand}}| \leq \Delta HR_{\text{max}}$$

$$|BR_k - BR_{\text{cand}}| \leq \Delta BR_{\text{max}}$$

where $\Delta HR_{\text{max}} = 8$ bpm and $\Delta BR_{\text{max}} = 3$ breaths/min. If a value exceeds the allowed jump, it is treated as a new candidate and must remain stable again before it is trusted. This prevents motion artifacts from immediately affecting the displayed output.

5. Exponential moving average

After a distance, heart-rate, or breathing-rate value passes the validation gates, it is smoothed using an exponential moving average (EMA):

$$y_k = y_{k-1} + \alpha(x_k - y_{k-1})$$

where x_k is the newest accepted value, y_k is the filtered output, and α is the smoothing factor. Since radar frames do not always arrive at fixed intervals, the smoothing factor is calculated using the elapsed time:

$$\alpha = \frac{\Delta t}{\tau + \Delta t}$$

where Δt is the time since the previous accepted sample and τ is the chosen time constant. Larger values of τ produce slower but smoother output. In the implementation, τ is set to 5 s for distance, 6 s for heart rate, and 6.5 s for breathing rate. This form makes the exponential moving average time-dependent: when the time between samples Δt is small, α becomes small and the filter changes slowly; when Δt is larger, α increases and the filter allows the new measurement to have more influence. This keeps the smoothing behaviour consistent even with irregular UART frame timing.

The EMA is suitable for live output because it acts as a lightweight first-order low-pass filter. It reduces fast fluctuations while still allowing slow real changes to appear.

6. Timeout reset

Each filter includes a timeout. If no valid input is received for a defined period, the corresponding filter is reset:

$$t_{\text{now}} - t_{\text{lastInput}} > T_{\text{timeout}}$$

This prevents old values from remaining trusted after the radar loses the target or stops producing valid measurements.

7. Final robust estimate

The live EMA output is useful for continuous monitoring, but it can still slowly drift during the first seconds of measurement. Therefore, an additional final-estimate stage is added. Once heart rate and breathing rate are both stable, the system collects one filtered sample per second for 20 seconds.

Let the stable filtered samples be:

$$x_1, x_2, \dots, x_N$$

The median is first calculated:

$$m = \text{median}(x_1, x_2, \dots, x_N)$$

Then the median absolute deviation (MAD) is calculated:

$$\text{MAD} = \text{median}(|x_i - m|)$$

The MAD is converted to a robust spread estimate:

$$\sigma_{\text{robust}} = 1.4826 \cdot \text{MAD}$$

Samples too far from the median are rejected:

$$|x_i - m| \leq k\sigma_{\text{robust}}$$

where $k = 2.5$. The final reported estimate is the mean of the remaining inlier samples:

$$\hat{x} = \frac{1}{M} \sum_{i=1}^M x_i$$

where M is the number of accepted samples. This method is applied separately to heart rate and breathing rate.

Justification of the method

The chosen method follows the principle of validation before smoothing. A simple moving average or EMA alone would reduce visible fluctuations, but it would still include false readings. This is not sufficient for this radar, because false presence and vital-sign frames may occur due to movement and reflections. Therefore, the algorithm first rejects unreliable samples using presence, distance, physiological, and stability constraints. Only after this validation are values smoothed.

The final estimate is based on a robust statistical approach. The median and MAD are less sensitive to outliers than the mean and standard deviation. This is suitable for radar vital-sign data because occasional false samples can still appear even after live filtering. The final estimate therefore represents the stable central tendency of the measurement window rather than the last instantaneous value.

This method is suitable for the Raspberry Pi Pico 2 W because it is computationally lightweight. It does not require storing large buffers or running complex signal-processing algorithms such as FFT or Kalman filtering. This makes it appropriate for filtering processed UART output from the radar module. The trade-off is increased latency. To a certain extent, this is acceptable because the goal of the system is to obtain stable and reliable vital-sign estimates rather than immediate response.

5

Results and Discussion

5.1. Radar Sensing Through Mirror Materials

The sensor module containing the MR60BHA2 radar sensor is placed within the smart mirror, which introduces three barriers between the radar antenna and the person whose vitals and presence are to be measured: an acrylic plate (with a thickness of 3mm), a layer of two-way mirror foil and a layer of paper. A test was conducted to reveal whether these added barriers influence the measurements produced by the MR60BHA2.

5.1.1. Openings for the Radar Sensor

To minimize the influence of the mirror assembly on radar performance, several cut-out geometries were evaluated in the acrylic mirror panel. The objective was to determine whether reducing the amount of material in front of the radar antenna could improve measurement accuracy while maintaining the visual appearance and structural integrity of the smart mirror.

The complete radar module measures 48×30 mm, containing a breakout circuit of the radar chip of 31.5×25 mm and an antenna array with dimensions of 21×17 mm (see figure 5.1). Since the antenna is responsible for transmitting and receiving electromagnetic waves, most cut-out designs were centered on the antenna region.

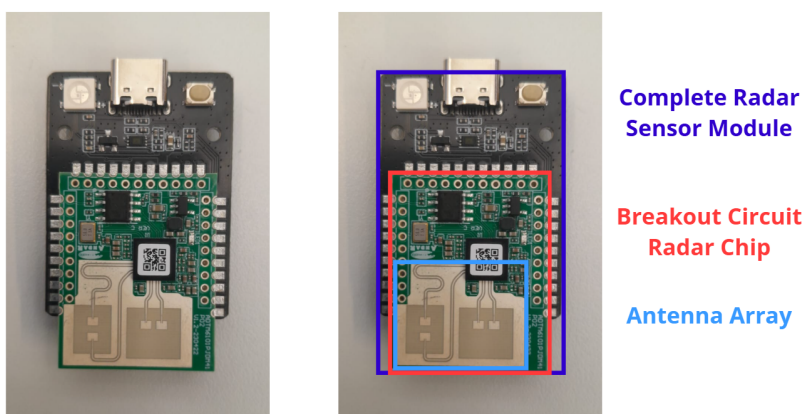


Figure 5.1: Different parts of the MR60BHA2 radar sensor module.

The investigated geometries included rectangular openings of various sizes, a circular opening, slot-based patterns, a perforated raster structure, and a geometry derived from the manufacturer's enclosure design. The selected designs ranged from minimal openings that exposed only the antenna area

to larger openings covering the complete radar module. See table 5.1 for an overview of the selected cut-outs and dimensions, and figure 5.2 for a visual overview.

Table 5.1: Evaluated cut-out geometries for the radar sensor

ID	Geometry	Dimensions	Reference Position
C1	Rectangle	21 × 17 mm	Aligned with antenna
C2	Rectangle	26 × 22 mm	Centered on antenna
C3	Rounded rectangle	31 × 27 mm, 5 mm fillet radius	Centered on antenna
C4	Rectangle	50 × 32 mm	Centered on complete PCB
C5	Circular opening	∅15 mm	Centered on antenna
C6	Rectangle	31 × 17 mm	Horizontally centered on antenna
C7	Manufacturer-inspired geometry	Based on supplied enclosure	Centered on antenna
C8	Two horizontal slots	25 × 6 mm, 3 mm spacing	Centered on antenna
C9	Perforated raster	4 mm holes, 2 mm ribs, total area 28 × 22 mm	Centered on radar chip

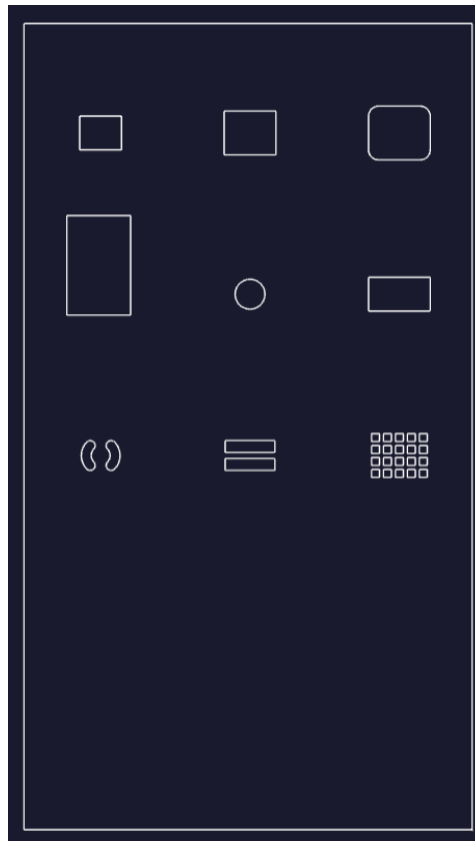


Figure 5.2: Evaluated cut-out geometries for the radar sensor, with c1 on the top left, and c9 on the bottom right.

5.1.2. Test Setup

The test setup consists of the MR60BHA2 module placed in a holder to prevent movement, with a person sitting directly in front of it (as in figure 5.3), at a fixed distance of 70cm. Any barriers were placed in front of the MR60BHA2, at a distance of 4mm.

The MR60BHA2 can measure distance to a person standing in front of the mirror, his/her respiratory rate and heart rate. To compare the effects of the different barriers, only the distance measurement was considered, as this is easy to validate with a tape measure, remain constant across different barrier tests, and because both the heart rate and respiratory rate values depend on the distance measured.

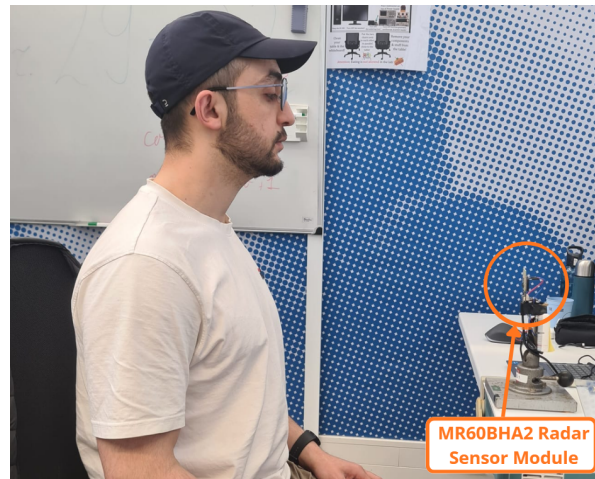


Figure 5.3: Image indicative of the test setup evaluating the effect of physical barriers on the measurements produced by the MR60BHA2 radar sensor module.

5.1.3. Results and Discussion

Table 5.2 shows the distance measurements obtained by the MR60BHA2 for several barriers. The measurements varied over time, which could partly be because of the movement of a person sitting in front of the sensor. Notably, no valid measurement data could be produced with the mirror foil as a barrier. This could be because mirror foil usually contains a thin metal layer, which reflects the electromagnetic waves and disturbs the measurements. Opening number 9 (a grid with 4mm square holes) also could not produce valid measurements.

From these measurements it can be concluded that an opening in the acrylic plate improves the accuracy of the measurements, and that several openings produce similar results. Which shape produces the best results and how much these differ from the actual distance cannot be determined from this test, due to the variations in the data.

Table 5.2: Distances measured by the MR60BHA2 for different barriers. The actual distance is 70cm.

Barrier	Measured distance (cm)
No Barrier	68-69
Mirror Foil	No Valid Data
Paper	63-65
Acrylic Plate (without opening)	63-65
Acrylic opening 1	66-70
Acrylic opening 2	63-65
Acrylic opening 3	66-70
Acrylic opening 4	66-70
Acrylic opening 5	66-70
Acrylic opening 6	66-70
Acrylic opening 7	66-70
Acrylic opening 8	66-70
Acrylic opening 9	No Valid Data

5.2. PCB Test and Validation

After the PCB was manufactured, a structured test and validation procedure was carried out to verify that the board functioned according to the design requirements. The validation focused on the power-supply section, microcontroller integration, communication interfaces, user-interface components, sensor connections, and the modular radar interface. The goal of this procedure was to confirm that the PCB work and also to identify possible design or assembly faults, incorrect connections, excessive voltage ripple or unstable communication.

The test procedure was divided into several stages. First, the PCB was inspected without power. Then, the microcontroller and individual interfaces were validated.

Test Equipment

The following laboratory equipment was used during the validation:

- Adjustable bench power supply with current limiting
- Digital multimeter
- Oscilloscope with 10x probes
- Resistors for loading the 5 V rail
- USB connection to the Raspberry Pi Pico 2WH for firmware upload and serial logging

Visual Inspection and Continuity Test

Before applying power, the PCB was visually inspected. The inspection focused on solder bridges, damaged pads, incorrect component orientation, connector alignment, and the polarity of diodes and electrolytic capacitors. Special attention was given to the power-supply section because incorrect polarity or a solder bridge in this area could damage the board during first power-up.

After the visual inspection, continuity measurements were performed using a multimeter. The resistance between $+5V_{SYS}$ and GND was measured to check for short circuits. The resistance between Vin_{BUCK} and GND was also checked before connecting the 12 V supply. In addition, the ground connections of all connectors were verified to confirm that the ground plane was correctly connected. Except for pin 16 of IC1 which is the GND pin for the C1001 module. The pin was inspected in kicad via Net Inspection Tool. From there it was evident that the copper connected with the pin was isolated from the GND net as shown in 5.4

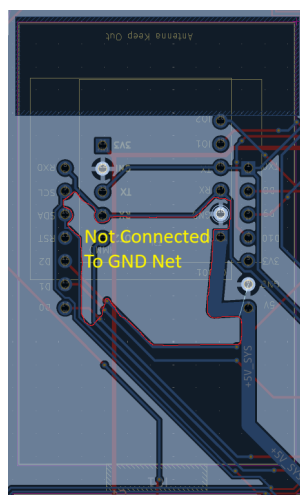


Figure 5.4: Pin 16

Power-Supply Validation

The first powered test was performed with the Raspberry Pi Pico and external sensor modules disconnected. The bench power supply was connected to the 12 V input and the current limit was set to a 100

Table 5.3: Unpowered PCB inspection and continuity test

Test	Expected result	Comments
Visual inspection	No solder bridges or damaged pads	Pass
Diode orientation	Matches schematic and silkscreen	Pass
Capacitor polarity	Correct orientation	Pass
V_{IN_BUCK} to GND	No short circuit	Pass
$+5V_{SYS}$ to GND	No short circuit	Pass
Ground pins	Connected to common GND	Only pin 16 was not connected

mA during initial connection. This reduces the risk of damaging the PCB in case of a short circuit or assembly fault. The current limit was increased gradually after the initial voltage measurements were confirmed.

Buck Converter Output Test

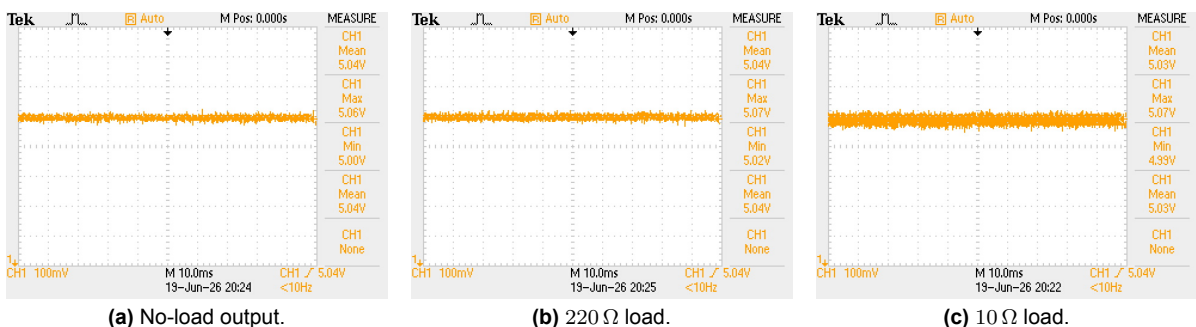
After assembling the buck converter circuit, the output voltage was tested using a 12 V input supply. The purpose of this test was to verify that the LM2596-based buck converter produces a stable 5 V output under no-load and light-load conditions. The output voltage was measured using both a multimeter and an oscilloscope. The oscilloscope was connected to the $+5V_{SYS}$ rail with the probe ground connected to PCB ground.

Three test conditions were used: no load, a $220\ \Omega$ load resistor, and a $10\ \Omega$ resistor. The $220\ \Omega$ corresponds to approximately 22.7 mA, while the $10\ \Omega$ load corresponds to approximately 0.5 A. The results are shown in Table 5.4.

Table 5.4: Measured buck converter output under different load conditions.

Test condition	Load	Estimated current	Multimeter V_{SYS}	Scope mean	Scope min	Scope max
No load	—	$\approx 0\text{ A}$	4.978 V	5.04 V	5.00 V	5.06 V
Light load	$220\ \Omega$	22.7 mA	4.988 V	5.04 V	5.02 V	5.07 V
Moderate load	$10\ \Omega$	500 mA	—	5.03 V	4.99 V	5.07 V

The measured output voltage remains close to the expected 5 V output in all three test cases. The oscilloscope measurements show only a small variation around the average output voltage. The approximate peak-to-peak variation, based on the measured maximum and minimum values, is around 60 mV with no load, 50 mV with the $220\ \Omega$ load, and 80 mV with the $10\ \Omega$ load. This indicates that the output remains stable under the tested load conditions.

**Figure 5.5:** Oscilloscope measurements of the 5 V system rail under different load conditions.

The oscilloscope traces appear as a thick horizontal line rather than a perfectly thin line because the measured signal contains a small amount of ripple and high-frequency noise. This is expected for a switching buck converter such as the LM2596. The regulator switches at a high frequency, so small switching ripple and noise components are present on the output voltage.

Overall, the test confirms that the buck converter produces a stable 5 V output from a 12 V input for the tested no-load and light/moderate-load conditions. The voltage stayed within an acceptable range and no abnormal voltage drop was observed. “

Microcontroller Power and Reset Validation

After the 5 V system rail was confirmed, the Raspberry Pi Pico 2WH was connected to the PCB. The 3.3 V rail generated by the Pico module was measured to confirm that the microcontroller and connected logic-level components were powered correctly. Since the GPIO pins of the Pico operate at 3.3 V logic level, the I2C, UART, button, and LED signals were checked to ensure that they did not exceed this voltage.

The reset button was validated by measuring the RUN pin. When the reset button is not pressed, the RUN pin should remain high. When the button is pressed, the RUN pin should be pulled to ground, causing the microcontroller to reset. This was verified using a multimeter.

Table 5.5: Microcontroller power and reset validation

Test item	Expected result	Measured result	Pass/Fail
Pico supply input	4.999 V	4.999 V	Pass
Pico 3.3 V rail	3.3 V	3.327 V	Pass
RUN pin idle state	Logic high	3.321 V	Pass
RUN pin when reset is pressed	Logic low	0 V	Pass
Microcontroller restart	Firmware restarts correctly	System Restarted	Pass

GPIO, LED, and Button Validation

A simple test firmware was uploaded to the Raspberry Pi Pico 2WH to validate the GPIO connections. The firmware toggled the GPIO-controlled LEDs and printed status messages over USB serial. This confirmed that the LEDs were connected to the correct GPIO pins and that the series resistors limited the LED current.

The display-control button was tested as a digital input. In the schematic, the button is configured as an active-low input. Therefore, the input should be high when the button is not pressed and low when the button is pressed. This was verified using the multimeter and by reading the button state in the Arduino IDE serial monitor. The code was programmed to print a message each time the button is pressed, High (released) to Low (pressed).

Table 5.6: GPIO, LED, and button validation

Function	Expected result	Measured result	Pass/Fail
Power LED	On when +5V _{SYS} is present	Didn't arrive	-
WiFi LED	Turn on when pin 19 = HIGH	Turned-On	Pass
Status LED	Turn on when pin 20 = HIGH	Turned-On	Pass
Display button idle state	Logic high	3.326 V	Pass
Display button pressed	Logic low	0 V	Pass

I2C Bus Validation

The status OLED display, environmental sensors and BH-1750 digital light sensor modules communicate with the microcontroller through the I2C bus. The first validation step was performed by placing the components and checking correct communication with the microcontroller. This was done by observing the values in the serial monitor and information depicted on the status OLED display. All modules behaved as expected where our reference is the state when we were testing with the breadboard.

The I2C signals were also measured using the multimeter at the SDA and SCL test points. The expected behaviour is that both lines are pulled high to approximately 3.3 V when idle and are pulled low during communication.

Table 5.7: I2C validation

Test item	Expected result	Measured result	Pass/Fail
SDA idle voltage	Approximately 3.3 V	3.326	Pass
SCL idle voltage	Approximately 3.3 V	3.326	Pass
status OLED display	Displays welcome message	"status OLED display"	Pass
Environmental sensor BMP280	Returns temperature and pressure values	21.3°, 1019.03 hPa	Pass

UART and Radar Interface Validation

The radar modules communicate with the microcontroller through UART. The HMMD radar module was connected and the UART signals were measured at the Tx and Rx test points. The multimeter was used to verify that the voltage levels were compatible with the microcontroller and that data activity was present. The firmware was then used to receive radar data. A successful test is indicated by stable UART communication and valid radar measurements.

Table 5.8: UART and radar interface validation

Test item	Expected result	Measured result	Pass/Fail
UART Tx voltage level	3.3 V logic	3.347	Pass
UART Rx voltage level	3.3 V logic	3.262	Pass
HMMD connected	Distance to target and energies in different gates	Results as expected	Pass

LDR and Analogue Input Validation

The light-dependent resistor was validated by measuring the analogue voltage at the ADC input while changing the light level. First, the LDR was covered to simulate a low-light condition. Then it was exposed to room light and finally to a stronger light source which was the flash light of a smartphone. The ADC value was read in the serial monitor and compared with the measured voltage using a multimeter.

As expected, the ADC voltage changes consistently with the light intensity. This confirms that the voltage divider is correctly connected and that the microcontroller can use the ADC input to estimate light intensity.

5.3. MR60BHA2 Validation

The MR60BHA2 radar was validated by comparing its filtered output with two reference measurements. Heart rate was compared with a Samsung Galaxy Watch 6, while breathing rate was compared with a manual counting method. For breathing rate, the number of visible chest movements was counted for 20 seconds.

The recorded measurements showed that the radar followed the same general pattern as the reference measurements, but a systematic difference was observed. For heart rate, the radar values were consistently higher than the Samsung Galaxy Watch values in all five trials. The differences ranged from

approximately 10.2 bpm to 14.2 bpm, with an average heart-rate bias of approximately 12.3 bpm. This indicates that the radar tended to overestimate the heart rate in the tested setup. Therefore, a preliminary correction offset was considered to reduce this systematic difference. However, this offset should be interpreted as an experimental calibration and not as a clinically validated correction, because the reference device was a consumer smartwatch rather than a medical-grade heart-rate monitor.

For breathing rate, the radar measurements were more consistent with the manual counting method than the heart-rate measurements. The differences ranged from approximately 1.4 to 2.5 breaths/min, with an average breathing-rate bias of approximately 1.8 breaths/min. Therefore, a preliminary offset of approximately -2 breaths/min was considered reasonable for this setup. Nevertheless, manual breathing-rate counting may introduce uncertainty, because the exact start and end of each breath can be difficult to determine consistently.

The active and relaxed measurements were also included to evaluate whether the radar could follow physiological trends. In the active condition, both the radar and the reference measurements showed an increase in heart rate compared with the normal-condition measurements. In the relaxed condition, both methods showed a decrease in heart rate and breathing rate. This suggests that, despite the observed absolute differences, the radar was able to follow the expected trend of increasing and decreasing vital signs. However, because only one active and one relaxed measurement were performed, the trend analysis should be interpreted as preliminary. More reliable validation would require repeated measurements and comparison with medical-grade reference devices.

Table 5.9: Measurement results for radar validation

Trial	Condition	Distance (cm)	Radar HR	Watch HR	Radar BR	Manual BR
1	Normal 1	49–50	67.2	57	19.7	18
2	Normal 2	45–46	70.9	58	18.4	17
3	Normal 3	80–82	73.1	60	25.0	23
4	Active	55–57	105.2	91	21.5	20
5	Relaxed	60–61	62.1	51	17.5	15

6

Conclusion

This thesis developed a concept demonstrator for a remote health monitoring system that supports elderly people living independently at home, observing presence, vital signs, and environmental conditions without requiring the user to wear, charge, or operate a device. The final design combines a modular sensor node, a smart mirror interface, a local MQTT broker, and a gateway that translates measurements into healthcare-compatible HL7 FHIR Observation resources, giving an end-to-end path from ambient sensing in the home to a healthcare information interface.

The main achievement is the integration of several technical domains into one coherent architecture. At the edge, the sensor module uses radar-based sensing for presence, heart rate, and respiratory rate, together with environmental and light sensors. The smart mirror presents the most relevant measurements through large icons, limited text, and a traffic-light scheme intended for elderly users with minimal interaction. The tablet also hosts the local MQTT broker, while the gateway subscribes to it, converts vital signs into LOINC-coded FHIR R4 Observations, and forwards them to a reference FHIR server, aligning the local data with existing healthcare IT standards.

A second achievement is the improved communication and security design. The system now uses a mutual-TLS architecture with static, factory-provisioned device certificates instead of the previous runtime-generated approach. The broker maps each certificate common name to a device identity, uses it as the MQTT username, and the access-control list confines each device to its own topic subtree. In the demonstrator, TLS 1.3 sessions, certificate-based identity, and per-device publish authorisation were confirmed from broker logs in the positive direction. Because Mosquitto enforces the access-control list at message delivery rather than at subscribe time, authorisation is observed by comparing what a device can publish against what it actually receives, not by inspecting subscribe acknowledgements. Structured negative testing of certificate-less connections, rogue-certificate-authority rejection, and cross-namespace isolation was not completed and remains verification work. For data loss during outages, the design specifies MQTT QoS 1 with acknowledgement, a non-volatile spool on the sensor node, and store-and-forward buffering on the gateway; QoS 1 and retained-message delivery were verified with reference MQTT clients, while the on-device spool and gateway buffer were not validated on production firmware.

The Programme of Requirements was met to a large extent at concept-demonstrator level. The smart mirror contains a sensor module, presents vital-sign and environmental information, and activates on presence detection. The sensor module integrates the required sensing, display, control, and communication components in a modular PCB design. The communication requirement was largely achieved through MQTT over mutual TLS, certificate-based authentication, and per-device publish authorisation, with negative-path authorisation testing and on-device outage buffering remaining as verification work. The healthcare integration requirement was demonstrated by translating measurements into FHIR R4 Observations and posting them over HTTPS to a reference FHIR server. Some requirements remain partially fulfilled: authentication against a production healthcare endpoint was not completed, the external infrastructure was tested only with a reference server, and the deployment automation still requires

hardening before reproducible installation in real homes.

The experimental results revealed important design lessons. The physical mirror construction strongly influences radar measurement quality: the mirror foil prevented valid measurements, most likely due to a reflective metallic layer, while openings in the acrylic plate improved distance readings, though not enough to determine one optimal geometry. Radar performance therefore cannot be treated as independent from the mechanical design; mirror material, foil, cut-out shape, and sensor placement must be designed together.

The MR60BHA2 validation showed the radar could follow expected physiological trends, such as a rise in heart rate after activity and a fall during relaxation. However, it consistently overestimated heart rate against the smartwatch reference, with an average bias of about 12.3 bpm, while breathing rate was closer to the manual reference at about 1.8 breaths/min. The radar is therefore promising for trend monitoring but not yet suitable for clinical decision-making without further calibration; medical-grade accuracy cannot be assumed from a commercial module.

The PCB validation confirmed that the custom hardware platform is a useful basis for further development. The microcontroller, reset circuit, I2C bus, OLED display, environmental sensor, UART radar interface, buttons, and status LEDs were tested and generally functioned as intended. The process also identified issues for a next revision, such as a missing ground connection on one connector pin and incomplete validation of the buck-converter power section due to component delivery delays.

Overall, the thesis demonstrates that a privacy-conscious, contactless, standards-aligned home monitoring system for elderly residents is feasible as a concept demonstrator. Such a system could support caregivers with routine observations of presence, vital signs, and environmental conditions while reducing the burden on the user. It is not a diagnostic or certified medical device, but a step toward safer independent living.

Future work should focus on the following main areas:

First, the radar measurements should be validated with medical-grade reference devices, more participants, repeated trials, and realistic standing positions in front of the mirror. A key recommendation is to move beyond ready-output commercial modules such as the C1001 or MR60BHA2 and investigate a dedicated radar platform from a manufacturer such as Texas Instruments, Infineon, or Acconeer. mmWave radar can detect the small chest movements caused by respiration and heartbeat without contact or a wearable [11], [15], but modules that internally process the signal output only final values, giving limited control over the pipeline. A future project should use a sensor exposing lower-level or raw data and build a processing pipeline tailored to a person standing at a defined distance from the mirror, accounting for posture, body movement, multipath reflections, and the mirror material. More advanced platforms achieve strong remote vital-sign estimation at higher algorithmic complexity [12], [14], [15]; this is substantial enough to anchor a future thesis.

The interface and overall product should also be validated through user testing with elderly participants, covering text readability, clarity of information, and general impressions. Finally, extended continuous-operation testing is needed to confirm the system runs reliably over long periods.

References

- [1] "Abstract colorful lines," Accessed: Jun. 16, 2026. [Online]. Available: <https://www.pexels.com/nl-nl/foto/abstract-abstracte-vormen-kleurrijk-lijnen-6604155/>.
- [2] "Statistics elderly people," Accessed: Jun. 16, 2026. [Online]. Available: <https://www.cbs.nl/en-gb/visualisations/dashboard-population/age/elderly-people>.
- [3] "Living independently for longer," Accessed: Jun. 16, 2026. [Online]. Available: <https://www.government.nl/themes/family-health-and-care/care-and-support-at-home/living-independently-for-longer>.
- [4] SmartHomeScene. "Seed studio 60 ghz mmwave sensors review," Accessed: May 5, 2026. [Online]. Available: <https://smarthomescene.com/reviews/seed-studio-60ghz-mmwave-sensors-review/>.
- [5] S. van der Avoird, Y. Rijkers, and R. van Leeuwen, "Hardware-driven (de-)registering peripheral devices for a medical system," Bachelor's thesis, Delft University of Technology, 2026.
- [6] N. Naik, "Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP," in *Proceedings of the IEEE International Symposium on Systems Engineering (ISSE)*, 2017. DOI: 10.1109/SysEng.2017.8088251.
- [7] J. Johnson and K. Finn, *Designing User Interfaces for an Aging Population: Towards Universal Design*. Morgan Kaufmann Publishers, 2017, ISBN: 978-0-12-804467-4. DOI: 10.1016/C2015-0-01451-4.
- [8] "Lvgl: Light and versatile embedded ui ecosystem," Accessed: Jun. 16, 2026. [Online]. Available: <https://lvgl.io/>.
- [9] "Dakboard," Accessed: May 12, 2026. [Online]. Available: <https://dakboard.com/site>.
- [10] "Magicmirror² docs," Accessed: May 8, 2026. [Online]. Available: <https://docs.magicmirror.builders/>.
- [11] Y. Wu, H. Ni, C. Mao, J. Han, and W. Xu, "Non-intrusive human vital sign detection using mmwave sensing technologies: A review," *ACM Transactions on Sensor Networks*, vol. 20, no. 1, Article 16, 2023, DOI: 10.1145/3627161.
- [12] Z. Hao, Y. Wang, F. Li, G. Ding, K. Fan, and Y. Gao, "Detection of vital signs based on millimeter wave radar," *Scientific Reports*, vol. 15, no. 1, p. 28 112, 2025, DOI: 10.1038/s41598-025-09112-w.
- [13] Y. Wang, W. Wang, M. Zhou, A. Ren, and Z. Tian, "Remote monitoring of human vital signs based on 77-ghz mm-wave fmcw radar," *Sensors*, vol. 20, no. 10, p. 2999, 2020, DOI: 10.3390/s20102999.
- [14] M. A. Nasser and L. Gromert. "Heart rate measurement using a 60 ghz pulsed coherent radar sensor," Accessed: May 5, 2026. [Online]. Available: <http://lup.lub.lu.se/student-papers/record/9094792>.
- [15] D. Wang, S. Yoo, and S. H. Cho, "Experimental comparison of ir-uwv radar and fmcw radar for vital signs," *Sensors*, vol. 20, no. 22, p. 6695, 2020, DOI: 10.3390/s20226695.
- [16] DFRobot. "Functional test report of dfrobot c1001 mmwave sensor (fall detection and sleep quality)," Accessed: May 5, 2026. [Online]. Available: <https://www.dfrobot.com/blog-13919.html>.
- [17] Seed Studio. "Mr60bha2 breathing and heartbeat module datasheet," Accessed: May 5, 2026. [Online]. Available: https://files.seeedstudio.com/wiki/mmwave-for-xiao/mr60/datasheet/MR60BHA2_Breathing_and_Heartbeat_Module.pdf.
- [18] DFRobot. "C1001 60ghz mmwave indoor fall detection sensor for arduino / esp32 / micro:bit (11 meters)," Accessed: May 5, 2026. [Online]. Available: <https://wiki.dfrobot.com/sen0623/>.

- [19] Seeed Studio. "Xiao 60ghz mmwave human breathing and heartbeat sensor -mr60bha2," Accessed: May 5, 2026. [Online]. Available: <https://www.seeedstudio.com/MR60BHA2-60GHz-mmWave-Sensor-Breathing-and-Heartbeat-Module-p-5945.html>.
- [20] Waveshare. "Hmmd mmwave sensor," Accessed: May 5, 2026. [Online]. Available: https://www.waveshare.com/wiki/HMMD_mmWave_Sensor.
- [21] Shenzhen Hi-Link Electronic Co., Ltd. "Hlk-lid2410 24ghz human presence body induction distance detection radar sensor module," Accessed: May 5, 2026. [Online]. Available: <https://www.hlktech.net/index.php?id=988>.
- [22] DFRobot. "Gravity: Mmwave c4001 24ghz sensor for precise human detection," Accessed: May 5, 2026. [Online]. Available: <https://wiki.dfrobot.com/sen0609/>.
- [23] Anonymous, "Performance evaluation of MQTT, CoAP and HTTP protocols for application-specific IoT scenarios," *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, vol. 14, no. II, Feb. 2026. DOI: 10.22214/ijraset.2026.77541.
- [24] Y. Demchenko, C. de Laat, D. R. Lopez, and J. A. García-Espin, "Zero-Touch provisioning (ZTP) model and infrastructure components for multi-provider cloud services provisioning," in *Proceedings of the IEEE International Conference on Cloud Engineering Workshop (IC2EW)*, 2016. arXiv: 1611.02758.
- [25] I. Boskov, H. Yetgin, M. Vučnik, C. Fortuna, and M. Mohorčič, "Time-to-provision evaluation of IoT devices using automated zero-touch provisioning," in *Proceedings of the IEEE Global Communications Conference (GLOBECOM)*, 2020. arXiv: 2009.09731.
- [26] M. Pritikin, M. Richardson, T. Eckert, M. Behringer, and K. Watsen, "Bootstrapping remote secure key infrastructure (BRSKI)," Internet Engineering Task Force (IETF), Request for Comments RFC 8995, 2021. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8995>.
- [27] H. Tschofenig and M. Pegourie-Gonnard, "Performance of state-of-the-art cryptography on ARM-based microprocessors," in *NIST Lightweight Cryptography Workshop*, 2015.
- [28] N. O. Gavrilidis, S. T. Halkidis, and S. Petridou, "Empirical evaluation of TLS-enhanced MQTT on IoT devices for V2X use cases," *Applied Sciences*, vol. 15, no. 15, p. 8398, 2025. DOI: 10.3390/app15158398.
- [29] K. Rzepka, P. Szary, K. Cabaj, and W. Mazurczyk, "Performance evaluation of raspberry Pi 4 and STM32 Nucleo boards for security-related operations in IoT environments," *Computer Networks*, vol. 242, p. 110 252, 2024. DOI: 10.1016/j.comnet.2024.110252.
- [30] Y.-C. Tsao, F.-J. Cheng, Y.-H. Li, and L.-D. Liao, "An IoT-based smart system with an MQTT broker for individual patient vital sign monitoring in potential emergency or prehospital applications," *Emergency Medicine International*, vol. 2022, p. 7 245 650, 2022. DOI: 10.1155/2022/7245650.
- [31] F. Famá, J. N. Faria, and D. Portugal, "An IoT-based interoperable architecture for wireless biomonitoring of patients with sensor patches," *Internet of Things*, vol. 19, p. 100 547, 2022. DOI: 10.1016/j.iot.2022.100547.
- [32] HL7 International. "FHIR release 4 (R4) specification," Accessed: May 5, 2026. [Online]. Available: <https://hl7.org/fhir/R4/>.
- [33] HL7 International. "FHIR vital signs profile," Accessed: May 5, 2026. [Online]. Available: <https://hl7.org/fhir/observation-vitalsigns.html>.
- [34] HL7 International. "Personal health device implementation guide v2.1.0," Accessed: May 5, 2026. [Online]. Available: <https://build.fhir.org/ig/HL7/phd/>.
- [35] F.-C. Adochiei, F.-A. Țoi, I.-R. Adochiei, F. C. Argatu, G. Serîțan, and G.-G. Petroiu, "HL7 FHIR-based open-source framework for real-time biomedical signal acquisition and IoMT interoperability," *Applied Sciences*, vol. 15, no. 23, p. 12 803, 2025. DOI: 10.3390/app152312803.
- [36] M. Marques, V. Delgado-Gomes, F. Januário, C. Lopes, R. Jardim-Goncalves, and C. Agostinho, "The B-Health Box: A standards-based fog IoT gateway for interoperable health and wellbeing data collection," *Sensors*, vol. 25, no. 23, p. 7116, 2025. DOI: 10.3390/s25237116.
- [37] DFRobot. "Gravity: Huskylens ai machine vision sensor (sen0305)," Accessed: May 5, 2026. [Online]. Available: <https://wiki.dfrobot.com/sen0305/>.

- [38] V. Bazarevsky, Y. Kartynnik, A. Vakunov, K. Raveendran, and M. Grundmann, *BlazeFace: Sub-millisecond neural face detection on mobile GPUs*, arXiv preprint arXiv:1907.05047, 2019. arXiv: 1907.05047 [cs.CV].
- [39] S. Chen, Y. Liu, X. Gao, and Z. Han, "MobileFaceNets: Efficient CNNs for accurate real-time face verification on mobile devices," in *Proceedings of the Chinese Conference on Biometric Recognition (CCBR)*, ser. Lecture Notes in Computer Science, vol. 10996, Urumqi, China: Springer, 2018, pp. 428–438. DOI: 10.1007/978-3-319-97909-0_46.
- [40] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 815–823. DOI: 10.1109/CVPR.2015.7298682.
- [41] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Long Beach, CA, USA, 2019, pp. 4690–4699. DOI: 10.1109/CVPR.2019.00482.
- [42] "Termux," Accessed: Jun. 15, 2026. [Online]. Available: <https://termux.dev/en/>.
- [43] "Fully kiosk browser," Accessed: Jun. 15, 2026. [Online]. Available: <https://www.fully-kiosk.com/>.
- [44] Sciosense. "Ens160 datasheet," Accessed: Mar. 29, 2023. [Online]. Available: <https://www.sciosense.com/wp-content/uploads/2023/12/ENS160-Datasheet.pdf>.
- [45] ASAIR. "Aht21 datasheet," Accessed: Jun. 8, 2026. [Online]. Available: <https://asairsensors.com/wp-content/uploads/2021/09/Data-Sheet-AHT21-Humidity-and-Temperature-Sensor-ASAIR-V1.0.03.pdf>.
- [46] Bosch. "Bmp280 datasheet," Accessed: Jun. 8, 2026. [Online]. Available: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp280-ds001.pdf>.
- [47] KiCad Project, *KiCad: Schematic Capture and PCB Design Software*, Official KiCad project website, 2026.
- [48] Texas Instruments, *LM2596 SIMPLE SWITCHER Power Converter 150-kHz 3-A Step-Down Voltage Regulator Datasheet*, SNVS124G, revised March 2023, 2023.
- [49] STMicroelectronics, *L78 Positive Voltage Regulator ICs Datasheet*, DS0422 Rev 38, February 2025, 2025.
- [50] "Linear versus switching regulators," Accessed: Jun. 12, 2026. [Online]. Available: <https://www.ti.com/lit/an/slyt527/slyt527.pdf?ts=1781518951061>.
- [51] F. Marinho, C. M. Carvalho, F. R. Apolinário, and L. Paulucci, "Measuring light with light dependent resistors: An easy approach for optics experiments," *European Journal of Physics*, vol. 40, no. 3, p. 035 801, 2019. DOI: 10.1088/1361-6404/ab11f1.

A

Visuals related to the Printed Circuit Board (PCB)

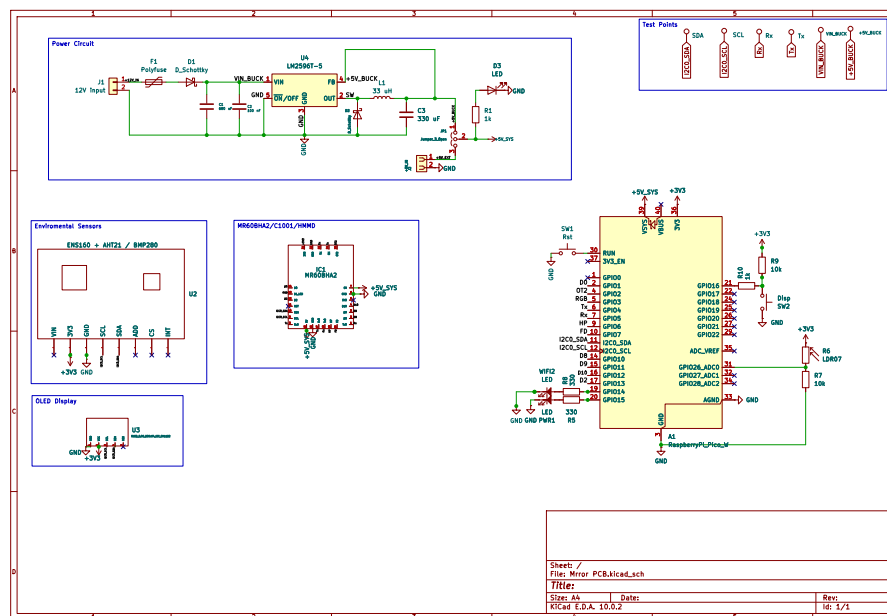
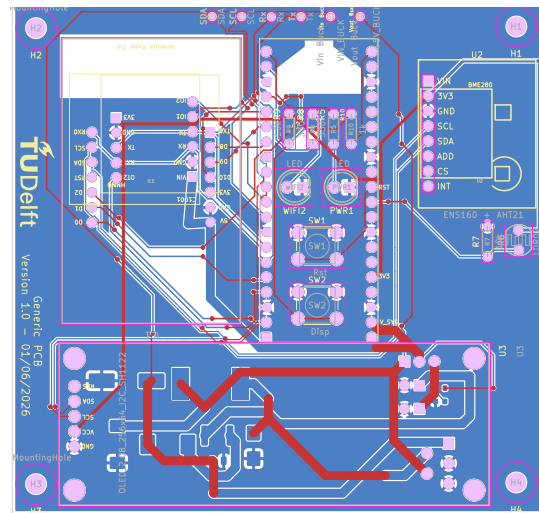
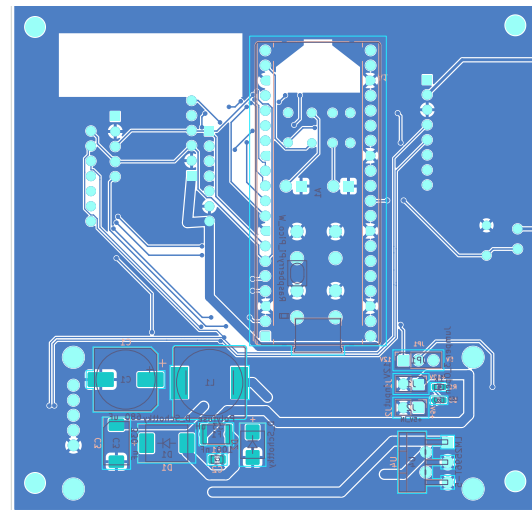


Figure A.1: PCB Schematic.

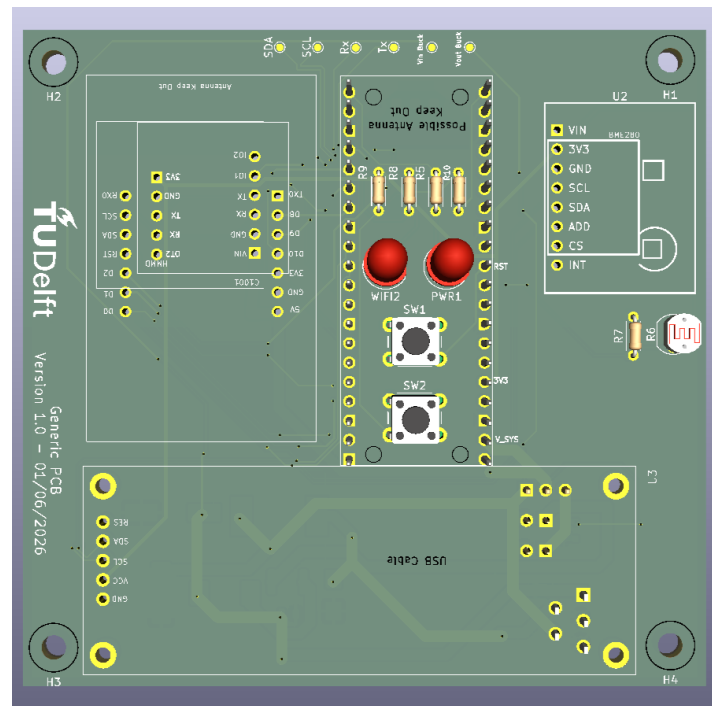


(a) PCB Layout: Front Side

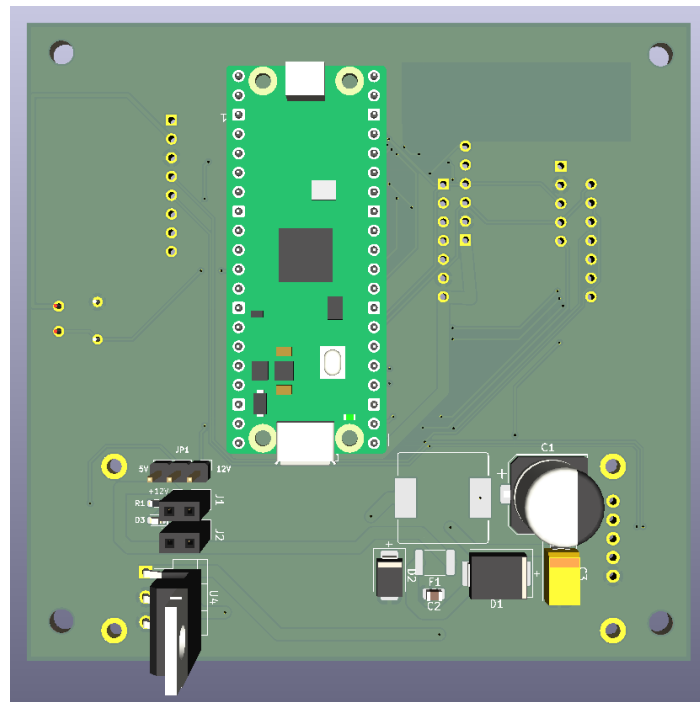


(b) PCB Layout: Back Side

Figure A.2: Complete KiCad PCB layout overview showing the front and back trace layouts.

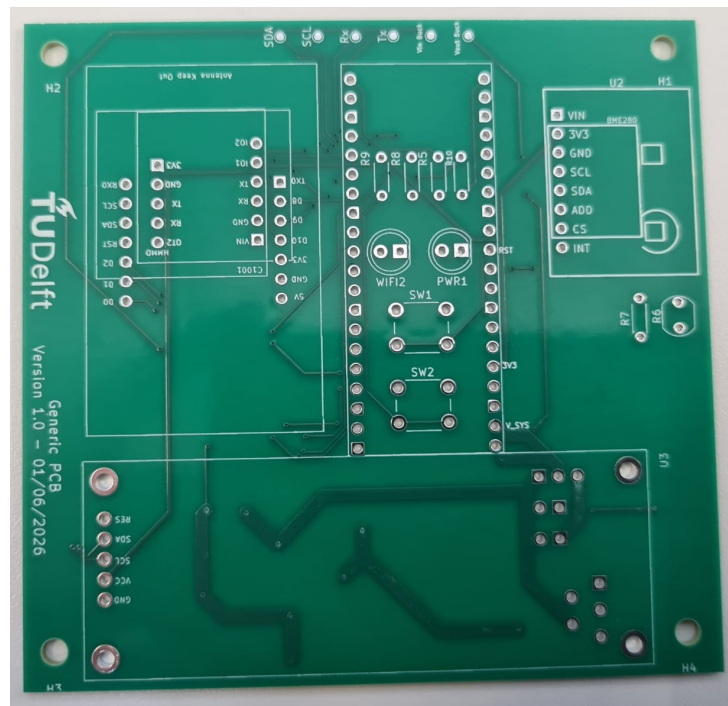


(a) PCB 3D View: Front Side

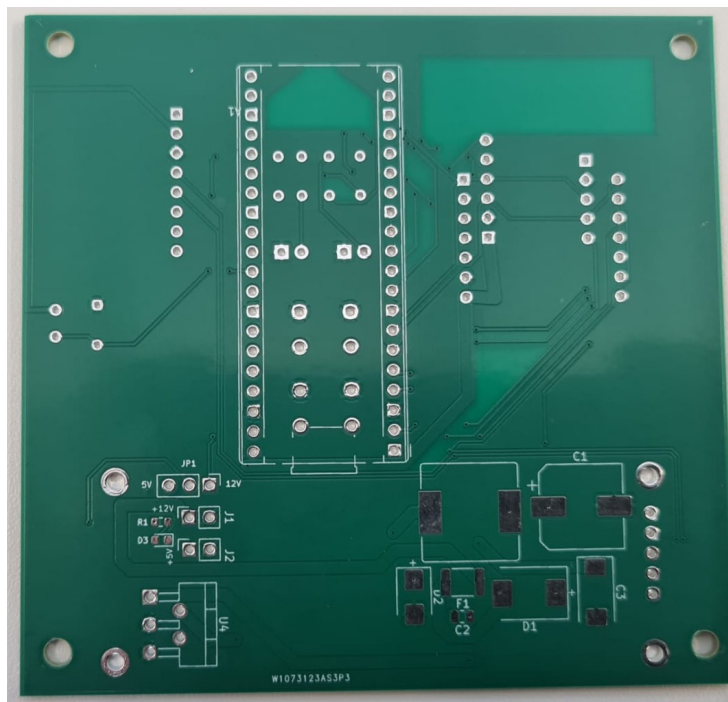


(b) PCB 3D View: Back Side

Figure A.3: Complete KiCad 3D hardware design overview showing the front and back sides of the PCB.

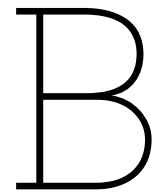


(a) PCB: Front Side



(b) PCB: Back Side

Figure A.4: Physical PCB overview showing the front and back sides.



Ethical Analysis

This chapter evaluates the ethical implications of the remote health monitoring system, addressing both the product and the engineering process behind it, as requested by the faculty.

B.1. Societal Impacts

The system targets a vulnerable group: elderly residents, many with cognitive decline, who are monitored continuously and passively in their own homes. The relevant stakeholders are these residents, their informal carers and family, the professional nursing staff and clinicians who act on the data, and the institutions that deploy and maintain the system. The central tension is privacy and autonomy against safety. Ambient sensing removes the burden of wearables and guarantees continuous data, but it constantly observes people who cannot easily perceive, refuse, or recall that observation. A second tension is justice: passive monitoring risks substituting surveillance for human presence for precisely those with the least bargaining power, and on cost grounds it could become a cheaper tier of attention rather than an addition to care.

B.2. Ethical Aspects of Development

The project builds upon a prior graduation thesis, which is attributed rather than silently reused [5]. Besides this, no other significant ethically relevant aspects of the development process were identified, as the project is mostly self-contained; we did not for example use or process data from a database or work with external test subjects.

B.3. Ethical Evaluation

The largest concern of the remote health monitoring system is the matter of privacy. Vital signs are sensitive data, and the choice of mmWave radar over cameras reduces intrusiveness while still capturing presence and vital signs. However, the measurement data obtained by the monitoring system is sent to a remote health care provider which can use pattern recognition to analyse activities of daily living. Whilst this is beneficial for the client in terms of medical health, it also forms a potential breach of privacy. The potential addition of cameras for facial recognition or other uses adds to this privacy concern. However, the system has the potential to provide remote health monitoring of elderly users for early detection of physiological complications and fast response in case of emergency, which promotes the well-being of the clients.

B.4. Reflection on Design Choices

A way in which the matter of privacy was addressed, was by encryption of the data communication to defend against unauthorized access. Binding each device's transport identity to its messaging identity and enforcing access control per device protects the confidentiality of sensitive data in transit, at the cost of a central trust root whose key custody becomes the single most sensitive parameter in the

system.

B.5. Forward-looking Responsibility

If the concept demonstrator were developed into a real-world healthcare application, measures should be taken to ensure ethical responsibility in both the system itself and the development process. The primary requirement is the protection of privacy and sensitive health information. Data collection should follow a principle of data minimization, where only information necessary for monitoring is collected and retained. Strong encryption for communication and storage of data should be maintained, together with strict access control so that only authorized healthcare professionals can access patient information.

Transparency and user autonomy should also be considered. Residents and their families should be informed about what data is collected, how it is used, and who has access to it. Since the target group includes elderly users with possible cognitive decline, special consideration should be given to informed consent procedures and involvement of carers where necessary.

Future development should also involve testing with healthcare professionals and representative users to identify unintended effects and usability issues. Finally, the system should support healthcare staff rather than replace human interaction, ensuring that efficiency gains do not come at the expense of quality of care.

C

Project Proposal

See next page for the proposal as provided by the supervisors.

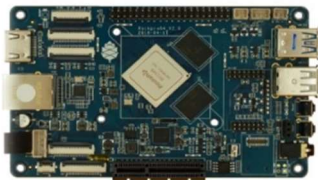
Assignment: Remote Health Monitoring System for the Elderly

Goal:

Design an innovative health-monitoring solution for elderly individuals that combines advanced sensor technology with embedded computing.

The system must use UWB radar for precise vital sign measurement, together with an environmental condition including temperature, humidity and air quality and a Single Board Computer (e.g., Pine RockPro64). It should provide remote monitoring while featuring a simple, user-friendly frontend tailored to seniors.

This assignment has been scaled to 3 students, who may build on the results of the prior project as their starting point!

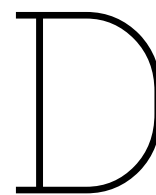


Project Objectives:

1. **Health Monitoring** – Integrate UWB radar sensors to capture heart rate, respiratory rate, and user positioning within living spaces. Optimize sensor placement for accuracy with minimal intrusion.
2. **Environment Sensing** – Measure temperature, humidity, and air quality.
3. **User Interface** – Develop a simple visual display (open-source format) that presents vital signs and environmental data in an easily understandable way for elderly users.

Deliverables:

- Redesigned current hardware (PCB layout) and embedded software that support the required sensor integration and user interaction.
- Revise the communication protocols chosen to balance reliability, security, and simplicity.
- Update deployment script including the API interfaces for seamless integration with existing medical infrastructure.



Tablet Deployment Instructions

Provisioning the tablet is more involved than a conventional Linux service installation for two reasons. First, Android is a difficult host for unattended background services: it has no service manager, applies aggressive power management (Doze), and may terminate background processes under memory pressure. Second, the system's security model forbids the project's certificate-authority (CA) private key from ever residing on the tablet. The procedure below is shaped by these two constraints.

D.0.1. A two-stage procedure

Provisioning is split into an on-device stage and a workstation stage. The on-device stage is a single, idempotent script (`deploy.sh`) run once inside Termux from a clone of the project repository; it installs and configures everything that can be done locally on the tablet. The workstation stage then runs from the developer laptop and pushes the tablet's mutual-TLS identity — its broker and mirror certificates together with the broker configuration — onto the device over SSH. This split follows directly from the security model: the CA private key is the root of trust for the entire device fleet, so it must not be placed on an always-on, physically accessible tablet. The tablet therefore receives only its own leaf certificates and the public CA certificate, and the on-device script handles no secret material.

D.0.2. On-device provisioning (`deploy.sh`)

The script is idempotent (safe to re-run) and performs the following:

1. Requests storage access and updates the Termux package index.
2. Installs the required packages:
 - **Git**, **Node.js** — MagicMirror² and its installer;
 - **Mosquitto** — the MQTT broker;
 - **OpenSSH** — remote access for the workstation stage;
 - **Python** with the `zeroconf` and `paho-mqtt` libraries — the mDNS responder and the presence-to-screen bridge;
 - the **Termux:API** tools — the wake-lock and the commands used to launch the kiosk browser.
3. Installs MagicMirror² and the MQTT-client dependency of the custom bridge module. Because the script runs from a Git checkout, MagicMirror's installer — which performs a clean-up step and fetches its bundled front-end assets — completes correctly.
4. Copies the runtime scripts into place and installs `tablet_boot.sh` as the Termux:Boot hook.
5. Starts the SSH daemon and prints the tablet's address together with the command to run on the workstation.
6. Stages the kiosk-browser configuration and the companion Android application in the Downloads folder for installation.

Until the certificates are delivered by the workstation stage, the mirror interface loads but the broker remains down; this is intentional, as the broker cannot start without its key material and the system does not fall back to an insecure mode.

D.0.3. Self-healing autostart (`tablet_boot.sh`)

Because Android offers no service manager, persistence is achieved with the *Termux:Boot* add-on, which runs a hook script after the device boots and is first unlocked. On each boot the hook executes, in order: acquire a wake-lock (to survive Doze); wait for Wi-Fi association; start the SSH daemon; start Mosquitto (once its certificates are present); start the mDNS responder; start the presence-to-screen bridge; and start MagicMirror² with its kiosk viewer. Each service is guarded so that re-running the hook never starts a duplicate, and the mirror launcher restarts the interface with exponential back-off on failure, so the tablet recovers from a reboot or power interruption without intervention.

D.0.4. Network-independent broker discovery

The sensor modules must locate the broker without hard-coded addresses, as the home network assigns IP addresses dynamically. The tablet advertises itself over multicast DNS under the stable name `tablet.local`; each sensor module is provisioned once with this name and resolves the current address at every connection. The installation therefore tolerates address changes and network moves without any re-provisioning of the sensor devices.

D.0.5. Manual steps (Android restrictions)

A few actions cannot be automated because of Android's security model and must be performed once by the installer:

- Disable battery optimisation for Termux (otherwise Doze suspends the services).
- Install the Termux:Boot add-on (from F-Droid).
- Unlock the device once after boot to arm the hook (it fires only after the first unlock).
- Import the kiosk-browser configuration and install the companion application when Android prompts.

D.0.6. Summarised procedure

```

1 On the tablet (once, in Termux):
2   1.  pkg install -y git
3   2.  git clone https://github.com/frederikschiepers123/BEP-Remote-Health-
4       Monitoring-System-for-the-Elderly
5   3.  cd BEP-Remote-Health-Monitoring-System-for-the-Elderly && ./deploy.sh
6
7 On the workstation (once):
8   4.  scripts/setup_tablet_ssh.sh          <tablet-ip>
9   5.  scripts/deploy_home_kit.sh tablet  <home-id> <tablet-ip>
10
11 Manual Android steps (once): disable battery optimisation for Termux,
install Termux:Boot, unlock after boot, import the kiosk configuration.
```

Listing D.1: Tablet deployment commands.